Vergleich von Regressionsverfahren im maschinellen Lernen

HILOMOT, LOLIMOT, MLP, SVR und GPR –
Theorie, Implementierung und Vergleich

anhand des LMNTool Multi GUI

Technische Dokumentation

21. April 2026

Kurzfassung

Diese technische Dokumentation gibt eine umfassende Einführung in fünf Regressionsverfahren des maschinellen Lernens: HILOMOT, LOLIMOT, MLP, SVR und GPR. Alle Methoden werden theoretisch hergeleitet, ihre Python-Implementierung in scikit-learn sowie der Bezug zur MATLAB LMNTool-Toolbox erläutert und anhand eines realen Datensatzes (Tilt Angle in Abhängigkeit von Scherkraft und Normalkraft) direkt verglichen. Die Dokumentation enthält Inhaltsverzeichnis, Abbildungsverzeichnis, Tabellenverzeichnis, Quellenangaben sowie eine Diskussion und Zusammenfassung.

Erstellt mit LaTeX und Biber/biblatex. Abbildungen aus dem LMNTool Multi GUI.

Inhaltsverzeichnis

Abbildungsverzeichnis **vi**

Tabellenverzeichnis **viii**

1 Einleitung **1**

1.1 Motivation und Hintergrund 1

1.2 Ziel und Aufbau der Dokumentation 1

1.3 Verwendeter Datensatz 2

2 Mathematische Grundlagen der Regression **4**

2.1 Was ist Regression? 4

2.2 Gütekriterien 4

2.2.1 Mittlerer quadratischer Fehler (MSE) 4

2.2.2 Mittlerer absoluter Fehler (MAE) 5

2.2.3 Wurzel des MSE (RMSE) 5

2.2.4 Bestimmtheitsmaß R^2 5

2.3 Kreuzvalidierung (Cross-Validation) 5

2.3.1 k-fache Kreuzvalidierung 5

2.3.2 Leave-One-Out Kreuzvalidierung (LOOCV) 6

2.4 Bias-Varianz-Dilemma 6

2.5 Normierung und Skalierung 6

3 Lokale Modellnetze – Grundlagen **7**

3.1 Motivation: Stückweise lineare Approximation 7

3.2 Mathematische Formulierung eines LMN 7

3.2.1 Partition of Unity 8

3.3 Gültigkeitsfunktionen und Sigmoid-Splits 8
3.4 Training der lokalen Modellparameter 8

4 LOLIMOT – Local Linear Model Tree 9

4.1 Einführung und historischer Hintergrund 9
4.2 Grundprinzip: Achsenparallele Splits 9
4.3 Algorithmus: LOLIMOT Tree Growing 10
4.4 Sigmoid-Gültigkeitsfunktionen und Smoothness 10
4.5 Bezug zur MATLAB LMNTool-Toolbox 10
4.6 Hyperparameter von LOLIMOT 11
4.7 Antwortfläche (LOLIMOT) 12
4.8 Stärken und Schwächen von LOLIMOT 12

5 HILOMOT – Hierarchical Local Model Tree 13

5.1 Einführung und Motivation 13
5.2 Oblique Splits: Mathematische Grundlage 13
5.3 Baumstruktur 14
5.4 Algorithmus: HILOMOT Tree Growing 14
5.5 Demanded Membership Value 15
5.6 Visualisierungen: Antwortfläche und Partitionierungskarte 15
5.7 Hyperparameter von HILOMOT 16
5.8 Stärken und Schwächen von HILOMOT 16

6 MLP – Multilayer Perceptron 17

6.1 Einführung und historischer Hintergrund 17
6.2 Architektur 17
6.3 Aktivierungsfunktionen 18
6.4 Backpropagation und Gradientenabstieg 18
6.5 Regularisierung: L2-Bestrafung 18
6.6 Universelle Approximationseigenschaft 19
6.7 Antwortfläche 19
6.8 Hyperparameter von MLPRegressor 20
6.9 Stärken und Schwächen von MLP 20

7 SVR – Support Vector Regression **21**

7.1 Einführung und historischer Hintergrund 21

7.2 Das ε-insensitive Verlustprinzip 21

7.3 Primäres Optimierungsproblem 21

7.4 Duales Problem und Kernel-Trick 22

7.5 RBF-Kernel 22

7.6 Antwortfläche 23

7.7 Hyperparameter von SVR 23

7.8 Stärken und Schwächen von SVR 24

8 GPR – Gaussian Process Regression **25**

8.1 Einführung und historischer Hintergrund 25

8.2 Grundkonzept: Verteilung über Funktionen 25

8.3 Kovarianzfunktionen (Kernel) 25

8.3.1 Squared Exponential (RBF) Kernel 26

8.3.2 Matérn-Kernel 26

8.3.3 Rauschterm 26

8.4 Bayes'sche Inferenz: Posterior-Verteilung 26

8.5 Hyperparameter-Optimierung via Marginal Likelihood 26

8.6 Antwortfläche 27

8.7 Hyperparameter von GaussianProcessRegressor 27

8.8 Stärken und Schwächen von GPR 28

9 LMNTool Multi GUI **29**

9.1 Überblick und Architektur 29

9.2 Datenimport und Vorverarbeitung 29

9.3 Methoden-Tabs und Parametrierung 30

9.4 Training und Bewertungslogik 30

9.5 Visualisierungen 31

9.6 Report- und Datenexport 32

10 MeinWindowLMNTool **34**

10.1 Überblick und Einordnung 34

10.2 Programmaufbau und Startvoraussetzungen 35
10.3 Datenimport 35
10.4 Methoden- und Parameterwahl 36
10.5 Training 36
10.6 Visualisierungen 37
10.7 Export und Integration 38
10.8 Vergleich mit dem Python LMNTool Multi GUI 39

11 Vergleich der Methoden **40**
11.1 Methodenüberblick 40
11.2 Vergleich der Antwortflächen 42
11.3 Tabellarischer Vergleich 43
11.4 Rechenkomplexität 43
11.5 Anwendungsempfehlungen 43

12 Diskussion **45**
12.1 Interpretation der Ergebnisse 45
12.2 Kritische Betrachtung der Hyperparameterwahl 45
12.3 Generalisierungsfähigkeit und Extrapolation 46
12.4 Grenzen des Vergleichs 46

13 Zusammenfassung **47**
13.1 Wichtigste Erkenntnisse 47
13.2 Praktische Handlungsempfehlungen 47
13.3 Ausblick 48

Abbildungsverzeichnis

1.1 3D-Ansicht der Trainingsdaten nach dem CSV-Import im LMNTool. Die Zielgröße y (Tilt Angle) ist als farbcodierte Höhe über den Eingangsgrößen x_1 (Shear Force) und x_2 (Normal Force) aufgetragen (blau = kleine Werte, rot = große Werte). 3

4.1 Modellierte Antwortfläche (LOLIMOT). Im Vergleich zu HILOMOT können achsenparallele Splits bei schräg orientierten Strukturen leicht höhere Stufenmäßigkeit erzeugen. Die Gültigkeitsfunktionen gewährleisten aber immer eine global glatte Fläche. 12

5.1 HILOMOT: Links die 3D-Vorhersagefläche, rechts die Partitionierungskarte. Jede Farbe entspricht einem dominanten lokalen Modell; die Linien zeigen die Trennhyperebenen. Oblique (schräge) Trennlinien sind für HILOMOT charakteristisch und ermöglichen eine effizientere Beschreibung diagonaler Strukturen als achsenparallele Schnitte. 15

6.1 Modellierte Antwortfläche (MLP). Das neuronale Netz approximiert die Zielfunktion als globale, glatte nichtlineare Abbildung ohne sichtbare Partitionierungsstruktur. Die Oberfläche ist weicher und weniger strukturiert als bei HILOMOT/LOLIMOT. 19

7.1 Modellierte Antwortfläche (SVR mit RBF-Kernel). Der Kernel ermöglicht die Approxi- mation komplexer Nichtlinearitäten bei gleichzeitig kontrollierter Komplexität durch C. 23

8.1 Modellierte Antwortfläche (GPR). Gaussian Process Regression liefert eine sehr glatte Approximation, die eng den Trainingsdaten folgt. In datenleeren Randbereichen zieht das Modell in Richtung der Priorfunktion. 27

9.1 3D-Datenvorschau im LMNTool nach dem CSV-Import. Die Visualisierung ermöglicht eine erste visuelle Plausibilitätsprüfung – z. B. auf Ausreißer, Lücken oder unerwartete Strukturen. 30

9.2 Korrelationsplot im LMNTool: gemessene Zielwerte y vs. Modellvorhersage $\hat{y}$. Gute Regression: Punkte nahe der Winkelhalbierenden. Systematische Abweichungen zeigen Modellbias. 31

9.3 Evaluation Bars: absoluter Vergleich aller fünf Methoden über die Kriterien Accuracy, Extrapolation Linearity, Numerical Efficiency und Interpretability. 31

9.4 Normalisierte Evaluation Bars: Jedes Kriterium ist auf $[0, 1]$ skaliert, wobei 1 der jeweils besten Methode entspricht. Diese Darstellung macht relative Stärken/Schwächen auf einen Blick erkennbar. 32

9.5 Tilt Angle Cuts: Schnitte durch die modellierten Antwortflächen bei sechs verschiedenen Normalkraft-Niveaus ($x_2 = \text{const}$). Alle fünf Methoden werden je Schnitt überlagert dargestellt. Abweichungen zwischen den Kurven zeigen methodenspezifische Unterschiede in Interpolation und Extrapolationsverhalten. 32

11.1 Vergleich der modellierten Antwortflächen aller fünf Methoden auf demselben Datensatz. Qualitative Unterschiede in Glätte, Partitionierungsstruktur und globalem Verhalten sind deutlich erkennbar. 42

Tabellenverzeichnis

2.1 Übersicht der Gütekriterien im Vergleich 5

4.1 Wichtige Hyperparameter von LOLIMOT im LMNTool 11

4.2 Stärken und Schwächen von LOLIMOT 12

5.1 Wichtige Hyperparameter von HILOMOT im LMNTool 16

5.2 Stärken und Schwächen von HILOMOT 16

6.1 Aktivierungsfunktionen für MLPs (Auswahl) 18

6.2 Wichtige Hyperparameter des MLP im LMNTool (`MLPRegressor`) 20

6.3 Stärken und Schwächen des MLP 20

7.1 Wichtige Hyperparameter der SVR im LMNTool 23

7.2 Stärken und Schwächen der SVR 24

8.1 Wichtige Hyperparameter der GPR im LMNTool 27

8.2 Stärken und Schwächen der GPR 28

9.1 Zusammenfassung der Kernparameter aller Methoden im LMNTool 30

10.1 Gegenüberstellung: MeinWindowLMNTool vs. LMNTool Multi GUI 34

10.2 Trainingsparameter in MeinWindowLMNTool 36

10.3 Visualisierungstypen in MeinWindowLMNTool 37

10.4 Exportformate in MeinWindowLMNTool 38

10.5 Funktionsumfang im Direktvergleich 39

11.1 Qualitative Einordnung der fünf Methoden nach Hauptcharakteristika 40

11.2 Ausführlicher Vergleich nach Stärken und Schwächen 43

11.3 Ungefähre Rechenkomplexität der fünf Methoden 43

Kapitel 1

Einleitung

1.1 Motivation und Hintergrund

Die Modellierung nichtlinearer Zusammenhänge zwischen Messgrössen ist eine der zentralen Aufgaben der modernen Ingenieurswissenschaften. In der Praxis werden physikalische Systeme – von Verbrennungsmotoren über Robotergelenke bis hin zu chemischen Reaktoren – durch Messdaten charakterisiert, aus denen ein mathematisches Modell abgeleitet werden soll. Dieses Vorgehen wird als *datengetriebene Modellbildung* oder *System Identification* bezeichnet [**nelles2001**].

Klassische lineare Modelle stoßen bei stark nichtlinearen Systemen schnell an ihre Grenzen. Auf der anderen Seite sind rein *black-box*-Verfahren wie tiefe neuronale Netze zwar leistungsfähig, bieten aber wenig Einblick in die innere Struktur – ein erheblicher Nachteil in sicherheits- oder normrelevanten Anwendungen.

Lokale Modellnetze (LMN) repräsentieren in diesem Spannungsfeld einen attraktiven Mittelweg: Sie zerlegen den Eingangsraum in Teilbereiche, in denen jeweils ein einfaches lineares Modell gilt. Der Ansatz ist interpretierbar und gleichzeitig flexibel genug für stark nichtlineare Kennlinien [**nelles2001**, **murray1998**].

Neben den LMN-Verfahren werden in dieser Dokumentation drei weitere etablierte Methoden untersucht: das Multilayer Perceptron (MLP) als Vertreter der neuronalen Netze [**goodfellow2016**], die Support Vector Regression (SVR) [**vapnik1998**] und die Gaussian Process Regression (GPR) [**rasmussen2006**]. Alle fünf Methoden sind im *LMNTool Multi GUI* implementiert und können direkt miteinander verglichen werden.

1.2 Ziel und Aufbau der Dokumentation

Diese Dokumentation verfolgt vier Hauptziele:

1. **Theorie:** Die mathematischen Grundlagen aller fünf Verfahren werden aus ersten Prinzipien hergeleitet, mit Bezug zu einschlägiger Fachliteratur und Online-Dokumentationen

(MATLAB [**matlab_gpr**, **matlab_svr**, **matlab_neural**], scikit-learn [**sklearn_mlp**, **sklearn_svr**, **sklearn_gpr**]).

2. **Implementierung:** Die konkrete Umsetzung in zwei Werkzeugen wird beschrieben: im Python-Tool *LMNTool Multi GUI* (fünf-Methoden-Vergleich) sowie in *MeinWindowLMNTool* (MATLAB-App-Designer-basiertes Trainingswerkzeug für HILOMOT/LOLIMOT), einschließlich aller relevanten Hyperparameter und ihrer Wirkung.

3. **Vergleich:** Alle Methoden werden auf demselben Datensatz trainiert und mit einheitlichen Gütekriterien verglichen.

4. **Empfehlung:** Auf Basis des Vergleichs werden Entscheidungshilfen für die Methodenwahl in der Praxis abgeleitet.

Der Aufbau der Dokumentation:

Kapitel 2 Mathematische Grundlagen der Regression: Gütekriterien, Kreuzvalidierung, Bias-Varianz-Dilemma.

Kapitel 3 Lokale Modellnetze – allgemeine Theorie (LMN, Partition of Unity, Gültigkeitsfunktionen).

Kapitel 4 LOLIMOT – achsenparallele Splits, Algorithmus, Hyperparameter.

Kapitel 5 HILOMOT – oblique Splits, Baumstruktur, Winkeloptimierung.

Kapitel 6 MLP – Architektur, Backpropagation, Aktivierungsfunktionen.

Kapitel 7 SVR – ε-Margin, Kernel-Trick, RBF-Kernel.

Kapitel 8 GPR – Gaußprozesse, Kovarianzfunktionen, Bayes'sche Inferenz.

Kapitel 9 LMNTool Multi GUI – Python-Vergleichswerkzeug, Datenimport, Parameter, Visualisierungen, Export.

Kapitel 10 MeinWindowLMNTool – MATLAB-App-Designer-Werkzeug für HILOMOT/LOLIMOT: Aufbau, Bedienung, Visualisierungen, Export und Einordnung.

Kapitel 11 Vergleich aller fünf Methoden.

Kapitel 12 Diskussion.

Kapitel 13 Zusammenfassung und Ausblick.

1.3 Verwendeter Datensatz

Der in dieser Dokumentation durchgängig verwendete Datensatz beschreibt den *Tilt Angle* (Neigungswinkel) in Abhängigkeit zweier Eingangsgrößen:

- x_1: **Shear Force** (Scherkraft)
- x_2: **Normal Force** (Normalkraft)
- y: **Tilt Angle** (Neigungswinkel, Zielgröße)

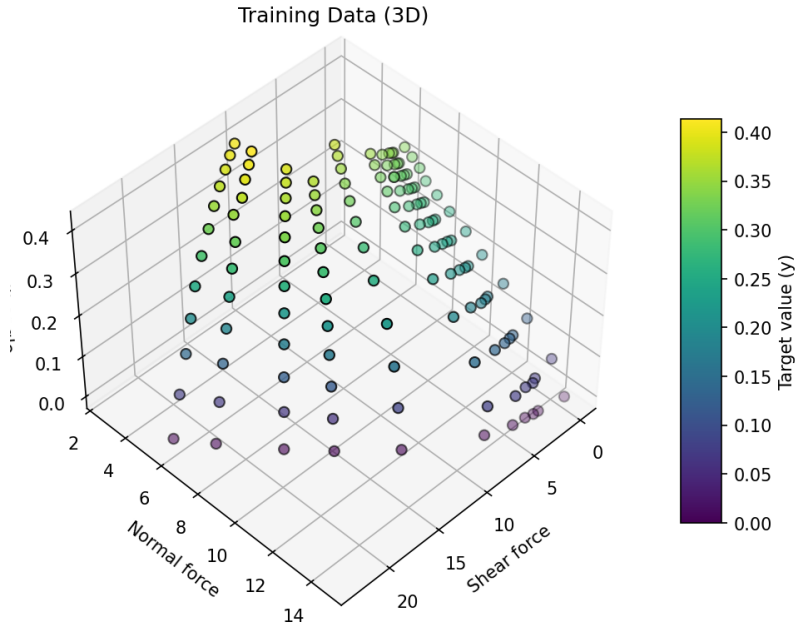


Abbildung 1.1: 3D-Ansicht der Trainingsdaten nach dem CSV-Import im LMNTool. Die Zielgröße y (Tilt Angle) ist als farbcodierte Höhe über den Eingangsgrößen x_1 (Shear Force) und x_2 (Normal Force) aufgetragen (blau = kleine Werte, rot = große Werte).

Der Datensatz weist eine glatte, nichtlineare Abhängigkeit der Zielgröße von beiden Eingangsgrößen auf und eignet sich damit gut für den Methodenvergleich. Die Daten wurden im LMNTool als CSV-Datei importiert (Spaltenreihenfolge: y, x_1, x_2) und für alle fünf Verfahren auf identischer Basis verwendet.

Kapitel 2

Mathematische Grundlagen der Regression

2.1 Was ist Regression?

Unter *Regression* versteht man die statistische Aufgabe, aus einem Datensatz $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ eine Funktion $\hat{f} : \mathbb{R}^n \to \mathbb{R}$ zu schätzen, die für neue Eingangsvektoren $\mathbf{x}^*$ eine möglichst genaue Vorhersage des zugehörigen Ausgangswerts y^* liefert [**bishop2006**].

Das allgemeine Regressionsmodell lautet:

$$y_i = f(\mathbf{x}_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \tag{2.1}$$

wobei ε_i das Messrauschen repräsentiert und als normalverteilt angenommen wird. Das Ziel ist, f durch $\hat{f}$ so zu approximieren, dass der Erwartungswert eines gewählten Verlustmaßes minimiert wird.

Die Komplexität des Regressionsmodells muss an die Komplexität der zugrunde liegenden Funktion f angepasst werden. Ist das Modell zu einfach, wird die Funktion nicht gut genügend beschrieben (*Underfitting*). Ist das Modell zu komplex, lernt es das Rauschen in den Trainingsdaten auswendig (*Overfitting*) [**hastie2009**].

2.2 Gütekriterien

Zur Bewertung der Modellgüte werden im LMNTool die folgenden standardisierten Maße verwendet [**hastie2009**, **bishop2006**]:

2.2.1 Mittlerer quadratischer Fehler (MSE)

$$\mathrm{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \tag{2.2}$$

Der MSE bewertet große Abweichungen überproportional (quadratisch). Er ist in den Einheiten $[y]^2$ angegeben, was die direkte Interpretation erschwert. Für Optimierung und Verlustfunktionen wird MSE jedoch häufig bevorzugt.

2.2.2 Mittlerer absoluter Fehler (MAE)

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \tag{2.3}$$

Der MAE ist robuster gegenüber Ausreißern als der MSE und in denselben Einheiten wie y.

2.2.3 Wurzel des MSE (RMSE)

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{2.4}$$

Der RMSE ist in denselben Einheiten wie y und damit intuitiver als der MSE.

2.2.4 Bestimmtheitsmaß R^2

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}, \qquad \bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i \tag{2.5}$$

$R^2 = 1$ bedeutet perfekte Anpassung; $R^2 = 0$ bedeutet, das Modell erklärt keine Varianz (gleich gut wie der Mittelwert). Negative Werte zeigen ein schlechteres Modell als der Mittelwert.

Tabelle 2.1: Übersicht der Gütekriterien im Vergleich

Maß	**Formel**	**Einheit**	**Optimal**
MSE	$\frac{1}{N}\sum(y_i - \hat{y}_i)^2$	$[y]^2$	$\to 0$
MAE	$\frac{1}{N}\sum\lvert y_i - \hat{y}_i\rvert$	$[y]$	$\to 0$
RMSE	$\sqrt{\mathrm{MSE}}$	$[y]$	$\to 0$
R^2	$1 - \mathrm{SS_{res}}/\mathrm{SS_{tot}}$	dimensionslos	$\to 1$

2.3 Kreuzvalidierung (Cross-Validation)

Gütekriterien auf den Trainingsdaten selbst sind oft zu optimistisch (Overfitting). Um die Generalisierungsfähigkeit eines Modells zu beurteilen, verwendet man *Kreuzvalidierung* [**hastie2009**, **sklearn_cross_validation**].

2.3.1 k-fache Kreuzvalidierung

Bei der k-fachen Kreuzvalidierung wird der Datensatz in k gleichgroße Teilmengen („Folds“) aufgeteilt. Das Modell wird k-mal trainiert: jeweils auf $k-1$ Folds, getestet auf dem verblei-

benden Fold:

$$\mathrm{CV}_k = \frac{1}{k}\sum_{j=1}^{k} \mathrm{Fehler}_j \tag{2.6}$$

Typisch sind $k = 5$ oder $k = 10$. Das LMNTool unterstützt k-fache Kreuzvalidierung zur Hyperparameterauswahl. scikit-learn stellt hierfür die Klasse `KFold` sowie `cross_val_score` bereit [**sklearn_cross_validation**].

2.3.2 Leave-One-Out Kreuzvalidierung (LOOCV)

LOOCV ist der Spezialfall $k = N$: Jeder Datenpunkt wird einmal als Testmenge verwendet. LOOCV ist rechenintensiv, aber für kleine Datensätze besonders geeignet, da keine Trainingsdaten verschwendet werden.

2.4 Bias-Varianz-Dilemma

Der erwartete Generalisierungsfehler lässt sich zerlegen in [**bishop2006**, **hastie2009**]:

$$\mathbb{E}\big[(y - \hat{f}(\mathbf{x}))^2\big] = \underbrace{\big(\mathrm{Bias}[\hat{f}]\big)^2}_{\text{Modellfehler}} + \underbrace{\mathrm{Var}[\hat{f}]}_{\text{Schätzunsicherheit}} + \underbrace{\sigma^2}_{\text{unvermeidbares Rauschen}} \tag{2.7}$$

Bias (Verzerrung): Systematischer Fehler des Modells; tritt auf, wenn das Modell zu einfach ist (*Underfitting*).

Varianz: Empfindlichkeit gegenüber Schwankungen in den Trainingsdaten; tritt auf, wenn das Modell zu komplex ist (*Overfitting*).

Merksatz

Die Kunst der Modellwahl liegt im richtigen Ausbalancieren von Bias und Varianz. Regularisierung (z. B. L2-Bestrafung bei MLPs, Margin bei SVR, Glättigkeitsparameter bei HILOMOT) ist das zentrale Mittel zur Varianzreduktion.

2.5 Normierung und Skalierung

Viele Verfahren (SVR, GPR, MLP) reagieren sensitiv auf die Größenordnung der Eingangsvariablen. Eine Standardisierung

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j} \tag{2.8}$$

mit Mittelwert μ_j und Standardabweichung σ_j der j-ten Eingangsdimension ist in der Praxis empfehlenswert. Das LMNTool normiert die Daten intern vor der Übergabe an die sklearn-Schätzer.

Kapitel 3

Lokale Modellnetze – Grundlagen

3.1 Motivation: Stückweise lineare Approximation

Die Idee der lokalen Modellnetze geht auf Jordan (1994) und Murray-Smith & Johansen (1997) zurück und wurde von Nelles (2001) für die Systemidentifikation weiterentwickelt [**nelles2001**, **murray1998**].

Grundprinzip: Eine komplexe nichtlineare Funktion wird nicht durch ein einziges globales Modell dargestellt, sondern durch eine Sammlung von einfachen, lokal gültigen Modellen. In jedem Teilbereich des Eingangsraums ist ein anderes Modell „zuständig“, und die Übergänge zwischen den Bereichen sind weich und glatt.

Analogie

Kartenblatt-Analogie: Eine globale Weltkarte in einer einzigen Projektion ist immer verzerrt. Navigationskarten bestehen dagegen aus vielen kleinen Blättern, die jeweils für einen lokalen Bereich eine genaue, nahezu lineare Projektion erlauben. Zusammen beschreiben sie die gekrümmte Erdoberfläche beliebig genau.

3.2 Mathematische Formulierung eines LMN

Ein lokales Modellnetz (LMN) approximiert eine Funktion $f : \mathbb{R}^n \to \mathbb{R}$:

$$\hat{y}(\mathbf{x}) = \sum_{i=1}^{M} \Phi_i(\mathbf{z})\, \hat{y}_i(\mathbf{x}) \tag{3.1}$$

mit:

- M: Anzahl der lokalen Modelle (Blätter),
- $\hat{y}_i(\mathbf{x}) = \boldsymbol{\theta}_i^T \tilde{\mathbf{x}}$: das i-te lokale lineare Modell mit $\tilde{\mathbf{x}} = [1, x_1, \ldots, x_n]^T$,
- $\Phi_i(\mathbf{z})$: Gültigkeitsfunktion des i-ten Modells,

- $\mathbf{z}$: Partitionierungsvektor (oft gleich $\mathbf{x}$).

3.2.1 Partition of Unity

Eine fundamentale Bedingung ist die *Partition-of-Unity-Eigenschaft*:

$$\sum_{i=1}^{M} \Phi_i(\mathbf{z}) = 1 \qquad \forall\, \mathbf{z} \tag{3.2}$$

Diese Bedingung stellt sicher, dass das Gesamtmodell in jedem Punkt eine konsistente, normierte Interpolation der lokalen Modelle ergibt. Bei baumstrukturierten Sigmoid-Splits wird sie automatisch erfüllt.

3.3 Gültigkeitsfunktionen und Sigmoid-Splits

An jedem inneren Knoten des Binärbaums wird der Eingangsraum durch eine weiche Sigmoid-Funktion aufgeteilt:

$$\Psi(\mathbf{z}) = \sigma\left(-\frac{\kappa}{\rho} \cdot (\mathbf{w}^T\mathbf{z} + b)\right), \qquad \sigma(x) = \frac{1}{1+e^{-x}} \tag{3.3}$$

mit Normalenvektor $\mathbf{w}$, Bias b, Schärfeparameter κ und Glättigkeitsparameter ρ (`smoothness`).

Das Gültigkeitsgewicht eines Blatts ergibt sich als Produkt der Sigmoid-Werte auf dem Pfad von der Wurzel:

$$\Phi_i(\mathbf{z}) = \prod_{k \in \mathrm{Pfad}_L(i)} \Psi_k(\mathbf{z}) \cdot \prod_{k \in \mathrm{Pfad}_R(i)} (1 - \Psi_k(\mathbf{z})) \tag{3.4}$$

Hinweis

Der Übergang zwischen zwei lokalen Modellen ist stets **weich und glatt** durch die Sigmoid-Funktion, was zu einem global glatten Gesamtmodell führt – ein wesentlicher Vorteil gegenüber harten Entscheidungsbäumen.

3.4 Training der lokalen Modellparameter

Die Parameter $\boldsymbol{\theta}_i$ der lokalen linearen Modelle werden durch **gewichtete kleinste Quadrate (WLS)** ermittelt:

$$\boldsymbol{\theta}_i = \left(\tilde{X}^T W_i \tilde{X}\right)^{-1} \tilde{X}^T W_i \mathbf{y} \tag{3.5}$$

mit $W_i = \mathrm{diag}(\Phi_i(\mathbf{z}_1), \ldots, \Phi_i(\mathbf{z}_N))$. Datenpunkte nahe der Mitte des i-ten Teilbereichs werden stärker gewichtet.

Kapitel 4

LOLIMOT – Local Linear Model Tree

4.1 Einführung und historischer Hintergrund

LOLIMOT (*Local Linear Model Tree*) wurde von Nelles und Isermann entwickelt und gehört zu den am häufigsten eingesetzten Verfahren zur Identifikation nichtlinearer dynamischer Systeme in der Regelungstechnik [**nelles2001**]. Das Verfahren kombiniert lokale lineare Modelle mit einem effizienten, greedy Tree-Growing-Algorithmus.

LOLIMOT ist auch in der MATLAB LMNTool-Toolbox implementiert, die als Referenz für das hier beschriebene Python-Tool dient [**matlab_lmnhelp**]. Die MATLAB-Dokumentation beschreibt die Grundlagen ebenfalls unter dem Stichwort „Local Linear Model Tree“ [**matlab_lmnhelp**].

4.2 Grundprinzip: Achsenparallele Splits

LOLIMOT ist ein Spezialfall des allgemeinen LMN-Rahmens (Kapitel 3). Die entscheidende Einschränkung ist, dass jeder Split **achsenparallel** erfolgt:

$$\mathbf{w} = \mathbf{e}_d, \qquad b = -c_d \tag{4.1}$$

wobei $\mathbf{e}_d$ der d-te Einheitsvektor und c_d den Schwellenwert entlang der d-ten Achse bezeichnet. Alle Trennlinien verlaufen entweder horizontal oder vertikal.

Analogie

Gitterpapier-Analogie: LOLIMOT unterteilt den Eingangsraum wie das Zeichnen von Linien auf Gitterpapier – alle Schnitte sind entweder waagrecht oder senkrecht. Diese Einschränkung macht das Verfahren einfach und robust, kann aber bei schrägen Strukturen zu unnötig vielen Teilregionen führen.

4.3 Algorithmus: LOLIMOT Tree Growing

LOLIMOT folgt einem greedy, inkrementellen Wachstumsalgorithmus [**nelles2001**]:

1. **Initialisierung:** Ein lineares Modell für den gesamten Eingangsraum. Parameter $\boldsymbol{\theta}_1$ per OLS (gewöhnliche kleinste Quadrate).
2. **Fehlerbestimmung:** Lokaler Fehler (gewichteter MSE) je Blatt:

$$e_i = \sum_{k=1}^{N} \Phi_i(\mathbf{z}_k) \left(y_k - \hat{y}_i(\mathbf{x}_k)\right)^2 \tag{4.2}$$

3. **Auswahl des schlechtesten Blatts** i^* mit größtem e_{i^*}.
4. **Split-Suche:** Für Blatt i^* alle achsenparallelen Kandidaten probieren (Dimension d, Schwellenwert c = Mittelpunkt des aktuellen Bereichs in Dim. d).
5. **Besten Split auswählen** (d^*, c^*) nach globalem Trainings-MSE.
6. **Modellupdate:** Blatt i^* durch neuen Knoten mit zwei Kindblättern ersetzen. WLS für neue Blattparameter (Gleichung 3.5).
7. **Abbruch** wenn `max_leafs` erreicht oder Fehler < Schwelle.

Hinweis

Der greedy Ansatz garantiert keine global optimale Lösung, ist aber in der Praxis sehr effektiv und schnell. Die Komplexität wächst mit $\mathcal{O}(M \cdot n \cdot N)$ pro Wachstumsschritt.

4.4 Sigmoid-Gültigkeitsfunktionen und Smoothness

Der Parameter `smoothness` (ρ) steuert, wie stark die Gültigkeitsbereiche benachbarter Teilmodelle überlappen:

- Kleines ρ: scharfe Trennung, wenig Überlappung – Gefahr von Unstetigkeiten.
- Großes ρ: glatter Übergang, starke Überlappung – Modell stets glatt, aber Trennstruktur kann verschmiert werden.

Typische Werte: $\rho \in [0{,}1,\ 0{,}5]$.

4.5 Bezug zur MATLAB LMNTool-Toolbox

Die MATLAB LMNTool-Toolbox [**matlab_lmnhelp**] bietet:

- Grafische Benutzeroberfläche (`lolimot_gui`),

- Skript-Schnittstellen (`lolimot()`, `hilomot()`),
- Export der gelernten Modellparameter,
- Visualisierungen der Gültigkeitsfunktionen und Partitionierungskarten.

Das Python-Tool *LMNTool Multi GUI* orientiert sich an dieser Schnittstelle.

4.6 Hyperparameter von LOLIMOT

Tabelle 4.1: Wichtige Hyperparameter von LOLIMOT im LMNTool

Parameter	**Typ**	**Beschreibung**
`max_leafs`	int	Max. Blätteranzahl; steuert Modellkomplexität.
`min_samples_per_leaf`	int	Mindestanzahl Datenpunkte je Blatt.
`smoothness` (ρ)	float	Überlappungsparameter der Sigmoid-Splits.
`demanded_membership`	float	Mindest-Gültigkeitswert für WLS-Schätzung.

4.7 Antwortfläche (LOLIMOT)

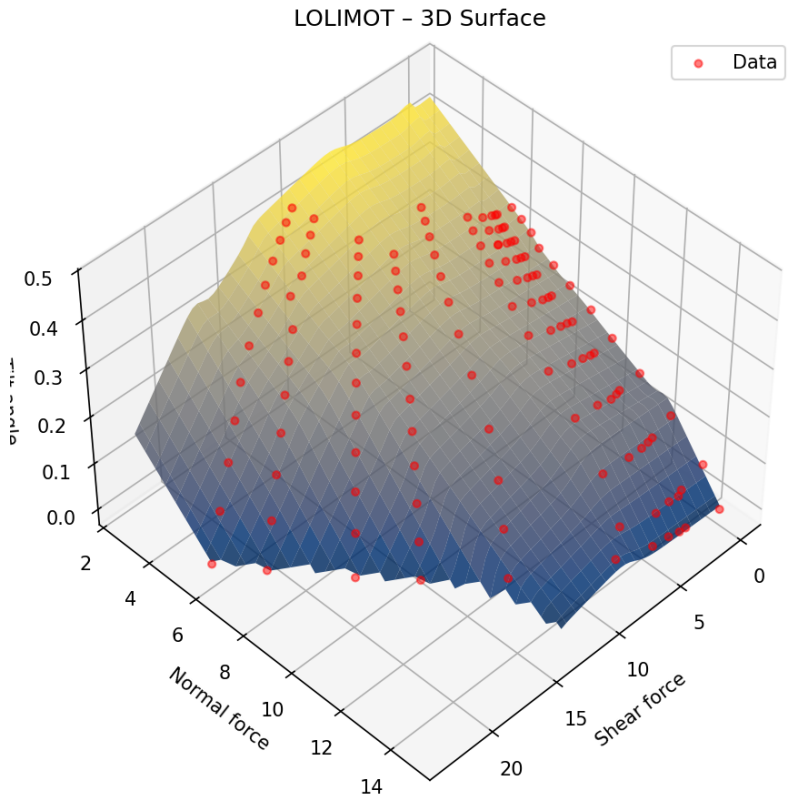


Abbildung 4.1: Modellierte Antwortfläche (LOLIMOT). Im Vergleich zu HILOMOT können achsenparallele Splits bei schräg orientierten Strukturen leicht höhere Stufenmäßigkeit erzeugen. Die Gültigkeitsfunktionen gewährleisten aber immer eine global glatte Fläche.

4.8 Stärken und Schwächen von LOLIMOT

Tabelle 4.2: Stärken und Schwächen von LOLIMOT

Stärken	**Schwächen**
Hohe Transparenz durch achsenparallele Regionen.	Treppenstufen-Problem bei schrägen Grenzen.
Robustes, stabiles Lernverfahren.	Keine diagonalen Trennlinien.
Einfache Kommunikation des Modells.	Geringere Effizienz bei gekoppelten Eingaben.
Geringe Rechenkosten.	Extrapolation über lokale Netze datenabhängig.

Kapitel 5

HILOMOT – Hierarchical Local Model Tree

5.1 Einführung und Motivation

HILOMOT (*Hierarchical Local Model Tree*) ist eine Erweiterung von LOLIMOT, die von Nelles vorgeschlagen wurde [**nelles2001**]. Die entscheidende Neuerung ist die Zulassung **oblique Splits** – Trennhyperebenen mit beliebiger Orientierung im Eingangsraum.

Merksatz

HILOMOT enthält LOLIMOT als Spezialfall: Wenn alle Trennrichtungen auf Einheitsvektoren $\mathbf{e}_d$ beschränkt werden, reduziert sich HILOMOT exakt auf LOLIMOT.

Der Vorteil oblique Splits: Wenn die Grenze zwischen zwei Betriebsbereichen im Kennfeld *diagonal* verläuft, kann LOLIMOT sie nur durch viele achsenparallele Schnitte annähern. HILOMOT kann sie direkt mit *einer* schrägen Trennebene erfassen – mit deutlich weniger Blättern.

5.2 Oblique Splits: Mathematische Grundlage

Ein HILOMOT-Split verwendet die allgemeine Hyperebenengleichung:

$$\mathbf{w}^T\mathbf{z} + b = 0 \tag{5.1}$$

mit beliebigem Normalenvektor $\mathbf{w} \in \mathbb{R}^n$ und Bias $b \in \mathbb{R}$.

Die zugehörige Sigmoid-Gültigkeitsfunktion:

$$\Psi(\mathbf{z}) = \frac{1}{1 + \exp\left(\frac{\kappa}{\rho}\left(\mathbf{w}^T\mathbf{z} + b\right)\right)} \tag{5.2}$$

Trennhyperebene unterteilt den Eingangsraum in zwei Halbräume:

- $\mathbf{w}^T\mathbf{z} + b < 0$: linker Teilbaum (Gewicht $\Psi \to 1$),
- $\mathbf{w}^T\mathbf{z} + b > 0$: rechter Teilbaum (Gewicht $\Psi \to 0$).

5.3 Baumstruktur

HILOMOT organisiert die Partitionierung als Binärbaum. Jeder innere Knoten enthält die Trennparameter $(\mathbf{w}, b, \rho)$, jedes Blatt die lokalen Modellparameter $\boldsymbol{\theta}_i$:

```
          [Knoten_1: w1, b1, rho1]
         /                        \
[Knoten_2: w2, b2, rho2]      Blatt_3: theta3
     /           \
Blatt_1: theta1  Blatt_2: theta2
```

5.4 Algorithmus: HILOMOT Tree Growing

Das Trainingsverfahren folgt dem greedy Wachstumsrahmen von LOLIMOT, erweitert um die Optimierung der Split-Richtung [**nelles2001**]:

1. **Initialisierung:** Wie LOLIMOT (ein Blatt).
2. **Fehlerbestimmung und Auswahl des schlechtesten Blatts.**
3. **Split-Kandidaten generieren:**
   - Alle achsenparallelen Richtungen $\mathbf{e}_1, \ldots, \mathbf{e}_n$.
   - Falls `oblique=True`: zusätzlich die erste Hauptkomponente (PCA-Richtung) der gewichteten Kovarianzmatrix des schlechtesten Blatts.
4. **Kontinuierliche Winkeloptimierung** (2D): Optimaler Winkel θ^*:

$$\theta^* = \arg\min_{\theta \in [0,\pi)} \mathrm{MSE}(\mathbf{y}, \hat{\mathbf{y}}(\theta)), \quad \mathbf{w}(\theta) = [\cos\theta, \sin\theta]^T \tag{5.3}$$

5. **Besten Split auswählen und Baum aktualisieren.**
6. **WLS für neue Blätter.**
7. **Abbruchkriterium prüfen.**

Hinweis

Die PCA-basierte Initialisierung der Splitrichtung ist ein zentraler Unterschied zu LOLIMOT. Sie liefert einen guten Startpunkt für die Winkeloptimierung und verbessert typischerweise die Lösungsqualität.

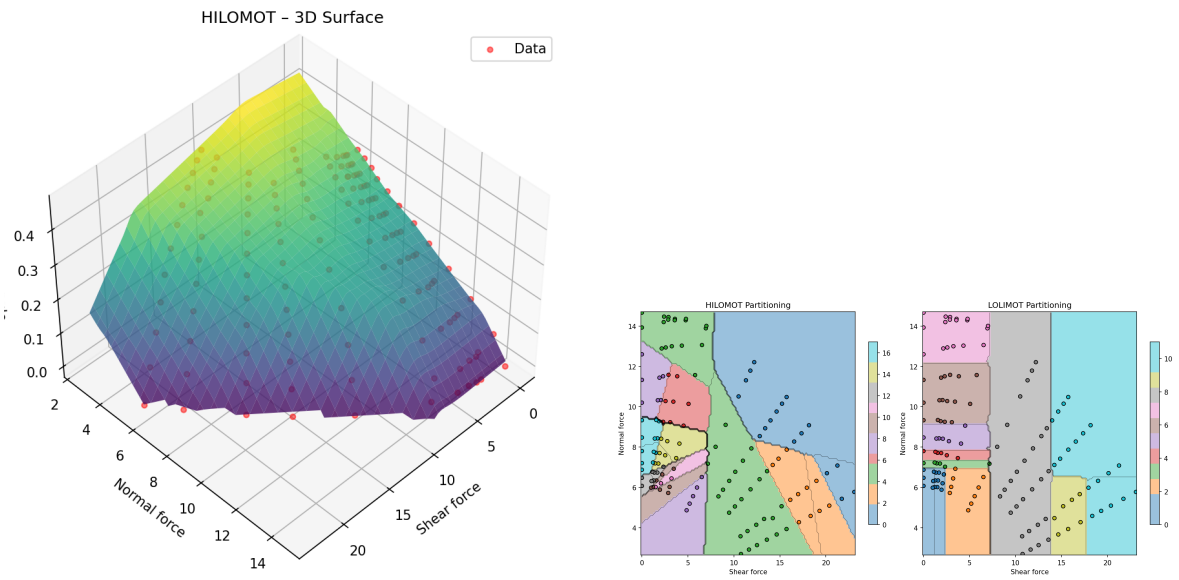
5.5 Demanded Membership Value

Der Parameter `demanded_membership_value` ($\mu_{\min}$) legt fest, ab welchem Gültigkeitswert ein Datenpunkt für die WLS-Schätzung eines Blatts verwendet wird:

$$\mathcal{D}_i^{\text{aktiv}} = \{k \mid \Phi_i(\mathbf{z}_k) \geq \mu_{\min}\} \tag{5.4}$$

Zu kleine Werte: Alle Datenpunkte beeinflussen alle Blätter – entspricht globalem Modell. Zu große Werte: Nur sehr nahe Datenpunkte – Risiko schlecht konditionierter WLS-Systeme.

5.6 Visualisierungen: Antwortfläche und Partitionierungskarte



(a) Antwortfläche (HILOMOT) (b) Partitionierungskarte (oblique)

Abbildung 5.1: HILOMOT: Links die 3D-Vorhersagefläche, rechts die Partitionierungskarte. Jede Farbe entspricht einem dominanten lokalen Modell; die Linien zeigen die Trennhyperebenen. Oblique (schräge) Trennlinien sind für HILOMOT charakteristisch und ermöglichen eine effizientere Beschreibung diagonaler Strukturen als achsenparallele Schnitte.

5.7 Hyperparameter von HILOMOT

Tabelle 5.1: Wichtige Hyperparameter von HILOMOT im LMNTool

Parameter	**Typ**	**Beschreibung**
`max_leafs`	int	Max. Blätteranzahl (Modellkomplexität).
`min_samples_per_leaf`	int	Mindestanzahl Datenpunkte je Blatt.
`smoothness` (ρ)	float	Steilheitsparameter der Sigmoid-Splits.
`demanded_membership`	float	Mindest-Gültigkeitswert für WLS-Schätzung.
`oblique`	bool	Aktiviert oblique Splits (PCA + Winkeloptimierung).

5.8 Stärken und Schwächen von HILOMOT

Tabelle 5.2: Stärken und Schwächen von HILOMOT

Stärken	**Schwächen**
Oblique Splits für effiziente Partitionierung bei gekoppelten Eingängen.	Training aufwändiger als LOLIMOT.
Lokale lineare Modelle physikalisch interpretierbar.	Greedy Wachstum ohne Globaloptimalitätsgarantie.
Oft weniger Blätter als LOLIMOT bei gleicher Genauigkeit.	Empfindlicher bei stark verrauschten Daten.
Glatte Vorhersagefläche durch Sigmoid-Übergänge.	
Dedizierter Parameterexport für Echtzeitsysteme.	

Kapitel 6

MLP – Multilayer Perceptron

6.1 Einführung und historischer Hintergrund

Das *Multilayer Perceptron* (MLP) ist die klassische Variante des vollständig verbundenen, vorwärtsgerichteten neuronalen Netzes. Die theoretischen Grundlagen gehen auf McCulloch & Pitts (1943) und Rosenblatt (1958) zurück; der entscheidende Durchbruch war die Entwicklung des Backpropagation-Algorithmus durch Rumelhart, Hinton und Williams (1986) [**rumelhart1986**, **goodfellow2016**, **haykin2009**].

Im LMNTool wird `MLPRegressor` aus scikit-learn verwendet [**sklearn_mlp**, **pedregosa2011**]. MATLAB stellt äquivalente Funktionalität über die Neural Network Toolbox und `fitrnet` (Statistics and Machine Learning Toolbox) bereit [**matlab_neural**].

6.2 Architektur

Ein MLP mit L Schichten besteht aus:

- einer **Eingabeschicht** (n Neuronen),
- $L-1$ **versteckten Schichten** mit je H_l Neuronen,
- einer **Ausgabeschicht** (1 Neuron für Regression).

Vorwärtsberechnung zwischen Schicht l und $l+1$:

$$\mathbf{h}^{(l+1)} = \sigma\Big(W^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}\Big) \tag{6.1}$$

mit Gewichtsmatrix $W^{(l)} \in \mathbb{R}^{H_{l+1}\times H_l}$ und Bias-Vektor $\mathbf{b}^{(l)} \in \mathbb{R}^{H_{l+1}}$. Die Ausgabe ohne Aktivierung:

$$\hat{y} = W^{(L)}\mathbf{h}^{(L)} + b^{(L)} \tag{6.2}$$

6.3 Aktivierungsfunktionen

Die Nichtlinearität des MLP entsteht durch die Aktivierungsfunktion σ. Tabelle 6.1 zeigt gängige Optionen.

Tabelle 6.1: Aktivierungsfunktionen für MLPs (Auswahl)

Name	**Formel**	**Bereich**	**Eigenschaft**
Sigmoid	$1/(1+e^{-x})$	$(0,1)$	Glatt, vanishing gradient
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	$(-1,1)$	Nullzentriert
ReLU	$\max(0,x)$	$[0,\infty)$	Einfach, kein vanishing gradient
Leaky ReLU	$\max(0.01x, x)$	$\mathbb{R}$	Verhindert dying ReLU
ELU	x für $x>0$; $\alpha(e^x-1)$ sonst	$\mathbb{R}$	Glatter Übergang

6.4 Backpropagation und Gradientenabstieg

Das MLP minimiert die Verlustfunktion MSE durch iterativen Gradientenabstieg [**rumelhart1986**, **goodfellow2016**]:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{6.3}$$

Backpropagation berechnet effizient $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ mittels der Kettenregel. Der einfache Gradientenabstieg:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta\nabla_{\boldsymbol{\theta}}\mathcal{L} \tag{6.4}$$

In scikit-learn ist auch **Adam** verfügbar [**sklearn_mlp**], der adaptive Lernraten pro Parameter verwendet:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \frac{\eta}{\sqrt{\hat{v}^{(t)}} + \epsilon}\,\hat{m}^{(t)} \tag{6.5}$$

mit laufendem Mittelwert $\hat{m}$ (erster Moment) und Varianzschätzung $\hat{v}$ (zweiter Moment).

6.5 Regularisierung: L2-Bestrafung

Die regularisierte Verlustfunktion:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\alpha}{2}\sum_{l}\|W^{(l)}\|_F^2 \tag{6.6}$$

α (`alpha`) groß: kleinere Gewichte, weniger Overfitting, mehr Bias.

6.6 Universelle Approximationseigenschaft

Ein MLP mit einer einzigen versteckten Schicht und genügend Neuronen kann jede stetige Funktion auf einem kompakten Definitionsbereich beliebig genau approximieren (*Universal Approximation Theorem*, Cybenko 1989) [**bishop2006**, **goodfellow2016**].

Hinweis

Die universelle Approximationseigenschaft garantiert *Existenz* einer Lösung, aber nicht *Auffindbarkeit* durch Gradientenabstieg. In der Praxis sind tiefere Netze mit weniger Neuronen je Schicht oft effizienter.

6.7 Antwortfläche

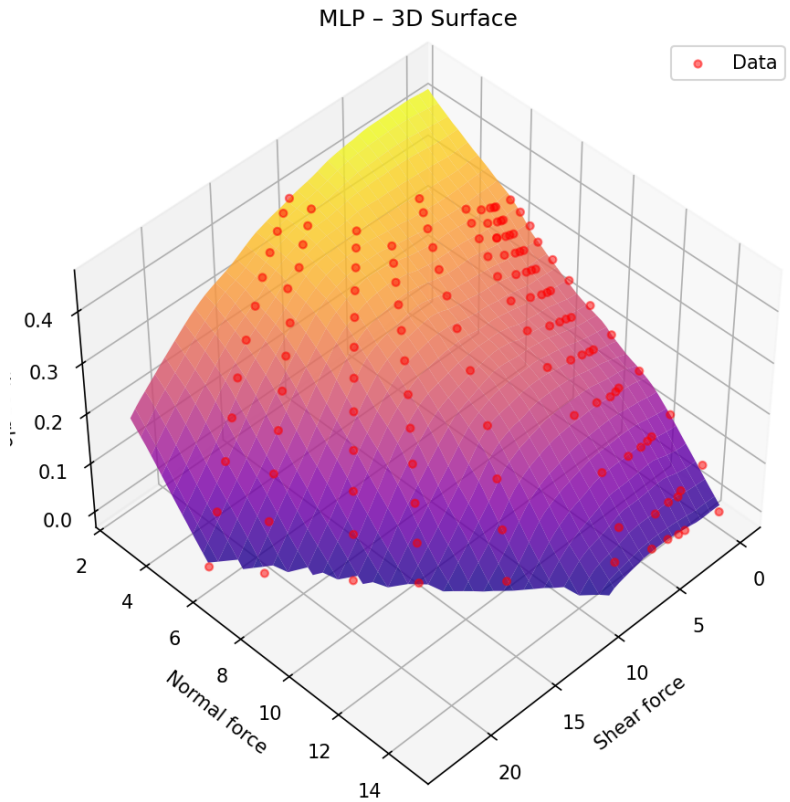


Abbildung 6.1: Modellierte Antwortfläche (MLP). Das neuronale Netz approximiert die Zielfunktion als globale, glatte nichtlineare Abbildung ohne sichtbare Partitionierungsstruktur. Die Oberfläche ist weicher und weniger strukturiert als bei HILOMOT/LOLIMOT.

6.8 Hyperparameter von MLPRegressor

Tabelle 6.2: Wichtige Hyperparameter des MLP im LMNTool (`MLPRegressor`)

Parameter	**Typ**	**Beschreibung**
`hidden_layer_sizes`	tuple	Neuronen je Schicht, z. B. `(100, 50)`.
`activation`	str	`'relu'` (Standard), `'tanh'`, `'logistic'`.
`alpha`	float	L2-Regularisierungsparameter.
`learning_rate_init`	float	Initialer Wert der Lernrate.
`max_iter`	int	Max. Trainingsiterationen (Epochen).
`solver`	str	`'adam'` (Standard), `'sgd'`, `'lbfgs'`.

6.9 Stärken und Schwächen von MLP

Tabelle 6.3: Stärken und Schwächen des MLP

Stärken	**Schwächen**
Sehr hohe funktionale Flexibilität.	Geringe Interpretierbarkeit (Black Box).
Starke Approximation beliebiger Nichtlinearitäten.	Sensitiv auf Hyperparameter.
Gute Fit-Werte bei ausreichenden Daten.	Overfitting-Risiko bei kleinen Datensätzen.
Skaliert gut mit Datenmenge.	Keine Unsicherheitsquantifizierung.
Breite Tooling-Unterstützung.	Skalierung der Eingaben essentiell.

Kapitel 7

SVR – Support Vector Regression

7.1 Einführung und historischer Hintergrund

Support Vector Machines wurden Anfang der 1990er Jahre von Vapnik entwickelt und für Regression als *Support Vector Regression* (SVR) adaptiert [**vapnik1998**, **drucker1997**, **smola2004**]. SVR gehört zu den leistungsfähigsten Verfahren für mittlere Datensätze.

MATLAB bietet SVR über `fitrsvm` im Statistics and Machine Learning Toolbox [**matlab_svr**]. Im LMNTool wird scikit-learn's `SVR` eingesetzt [**sklearn_svr**, **pedregosa2011**].

7.2 Das ε-insensitive Verlustprinzip

Im Gegensatz zum MSE-Verlust definiert SVR eine ε-*Tube* um die Vorhersage: Fehler betragsmäßig kleiner als ε erhalten Verlust 0:

$$L_\varepsilon(y, \hat{y}) = \max(0,\, |y - \hat{y}| - \varepsilon) \tag{7.1}$$

Nur Datenpunkte außerhalb der ε-Tube (die *Support Vectors*) bestimmen das Modell – eine sparsame Darstellung.

7.3 Primäres Optimierungsproblem

Das primäre SVR-Problem (lineare Variante) [**vapnik1998**, **smola2004**]:

$$\min_{\mathbf{w}, b, \xi, \xi^*} \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{N} (\xi_i + \xi_i^*) \tag{7.2}$$

unter Nebenbedingungen:

$$y_i - \mathbf{w}^T\mathbf{x}_i - b \leq \varepsilon + \xi_i \tag{7.3}$$

$$\mathbf{w}^T\mathbf{x}_i + b - y_i \leq \varepsilon + \xi_i^* \tag{7.4}$$

$$\xi_i, \xi_i^* \geq 0 \tag{7.5}$$

$C > 0$ ist der Regularisierungsparameter; ξ_i, ξ_i^* Schlupfvariablen.

Einfluss von C:

- Kleines C: viele Support Vectors erlaubt, Modell glatter (hoher Bias, geringe Varianz).
- Großes C: wenige Ausreißer toleriert, Modell kann komplexer werden.

7.4 Duales Problem und Kernel-Trick

Das duale Problem:

$$\max_{\alpha,\alpha^*} \; -\frac{1}{2}\sum_{i,j}(\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*)K(\mathbf{x}_i, \mathbf{x}_j) - \varepsilon\sum_i(\alpha_i + \alpha_i^*) + \sum_i y_i(\alpha_i - \alpha_i^*) \tag{7.6}$$

Vorhersage für neuen Punkt $\mathbf{x}^*$:

$$\hat{y}(\mathbf{x}^*) = \sum_{i \in \mathrm{SV}} (\alpha_i - \alpha_i^*)\, K(\mathbf{x}_i, \mathbf{x}^*) + b \tag{7.7}$$

Kernel-Trick: Durch $K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$ wird implizit in einen höherdimensionalen Merkmalsraum eingebettet [**scholkopf2002**].

7.5 RBF-Kernel

Der am häufigsten verwendete Kernel für nichtlineare SVR:

$$K(\mathbf{x}, \mathbf{x}') = \exp\Big(-\gamma \, \|\mathbf{x} - \mathbf{x}'\|^2\Big) \tag{7.8}$$

γ steuert die Reichweite des Kernels: Kleines γ = breiter Kern, global wirkend; großes γ = schmaler Kern, lokal wirkend.

Hinweis

Für die meisten Regressionsprobleme mit unbekannter Datenstruktur ist der RBF-Kernel eine gute Standardwahl. MATLAB empfiehlt ebenfalls den RBF-Kernel als Standardoption für `fitrsvm` [**matlab_svr**].

7.6 Antwortfläche

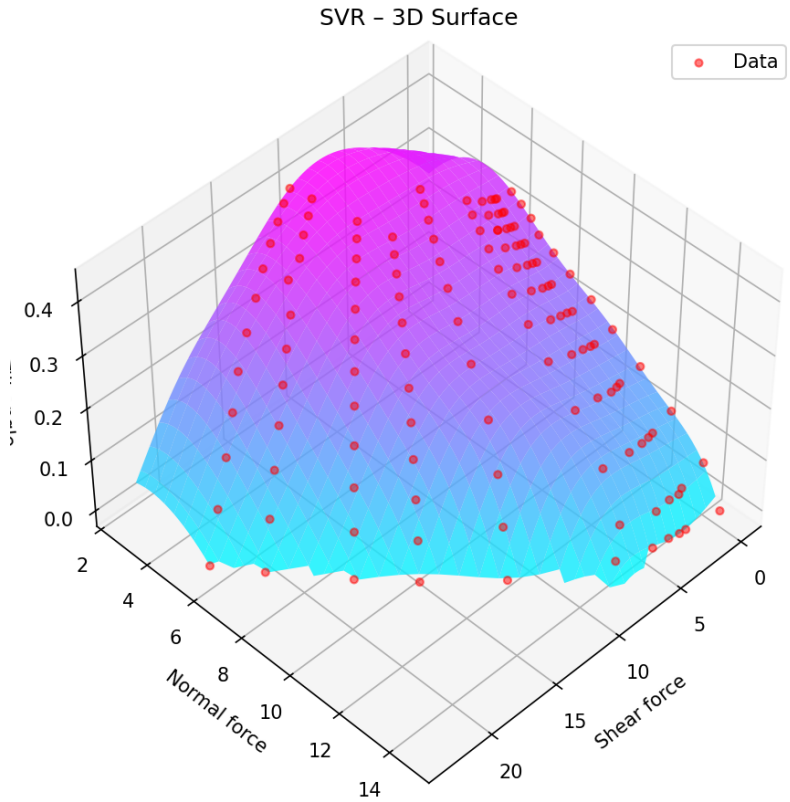


Abbildung 7.1: Modellierte Antwortfläche (SVR mit RBF-Kernel). Der Kernel ermöglicht die Approxi- mation komplexer Nichtlinearitäten bei gleichzeitig kontrollierter Komplexität durch C.

7.7 Hyperparameter von SVR

Tabelle 7.1: Wichtige Hyperparameter der SVR im LMNTool

Parameter	Typ	Beschreibung
`C`	float	Regularisierungsparameter. Groß: enger Fit. Klein: glattest.
`gamma`	float	RBF-Kernel-Reichweite. Groß: lokal. Klein: global.
`epsilon`	float	Breite der ε-Tube. Klein: engere Toleranz, mehr SVs.
`kernel`	str	`'rbf'` (Standard), `'poly'`, `'sigmoid'`.

7.8 Stärken und Schwächen von SVR

Tabelle 7.2: Stärken und Schwächen der SVR

Stärken	**Schwächen**
Gute Generalisierung bei kleinen bis mittleren Datensätzen.	Parameter schwer intuitiv einzustellen.
Robustes Lernprinzip durch Margin-Maximierung.	Hohe Rechenkosten für große N.
Sparsame Darstellung durch Support Vectors.	Kein probabilistisches Ausgabeformat.
Guter RBF-Kernel für unbekannte Strukturen.	Schlechte Interpretierbarkeit.

Kapitel 8

GPR – Gaussian Process Regression

8.1 Einführung und historischer Hintergrund

Gaussian Process Regression (GPR) – auch *Kriging* in der Geostatistik – ist ein probabilistisches, nichtparametrisches Verfahren [**rasmussen2006**]. Im Gegensatz zu HILOMOT/LOLIMOT, MLP und SVR gibt GPR eine *vollständige Wahrscheinlichkeitsverteilung* über mögliche Funktionswerte aus, also neben der Punktvorhersage auch Unsicherheitsintervalle.

MATLAB bietet GPR über `fitrgp` [**matlab_gpr**]. Im LMNTool wird scikit-learn's `GaussianProcessRegres` verwendet [**sklearn_gpr**, **pedregosa2011**].

8.2 Grundkonzept: Verteilung über Funktionen

Ein *Gaußscher Prozess* ist eine Verallgemeinerung der multivariaten Normalverteilung auf Funktionen:

$$f(\mathbf{x}) \sim \mathcal{GP}\big(m(\mathbf{x}),\, k(\mathbf{x}, \mathbf{x}')\big) \tag{8.1}$$

mit Mittelwertfunktion $m(\mathbf{x})$ und Kovarianzfunktion $k(\mathbf{x}, \mathbf{x}')$.

Für endlich viele Punkte $\mathbf{X}$:

$$\mathbf{f} \sim \mathcal{N}(\mathbf{m},\, K(\mathbf{X}, \mathbf{X})) \tag{8.2}$$

mit Gram-Matrix $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.

8.3 Kovarianzfunktionen (Kernel)

Die Kernelwahl bestimmt die a-priori-Annahmen über Glattheit und Struktur der Funktion [**rasmussen2006**]:

8.3.1 Squared Exponential (RBF) Kernel

$$k_{\mathrm{SE}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) \tag{8.3}$$

ℓ (Length-Scale): Korrelationsreichweite; σ_f^2 (Signal-Varianz): Schwankungsbreite der a-priori-Funktionen.

8.3.2 Matérn-Kernel

$$k_\nu(r) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}\, r}{\ell}\right)^\nu K_\nu\left(\frac{\sqrt{2\nu}\, r}{\ell}\right) \tag{8.4}$$

mit $r = \|\mathbf{x} - \mathbf{x}'\|$, modifizierter Bessel-Funktion K_ν und Glättegrad ν. Für $\nu \to \infty$ ergibt sich der SE-Kernel.

8.3.3 Rauschterm

$$k_{\mathrm{ges}}(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}, \mathbf{x}') + \sigma_n^2\, \delta(\mathbf{x}, \mathbf{x}') \tag{8.5}$$

mit Rauschvarianz σ_n^2 (`noise_level`).

8.4 Bayes'sche Inferenz: Posterior-Verteilung

Gegeben Trainingsdaten $\mathcal{D} = (\mathbf{X}, \mathbf{y})$ und neuer Punkt $\mathbf{x}^*$ [**rasmussen2006**]:

$$\hat{f}(\mathbf{x}^*) = \mathbf{k}_*^T (K + \sigma_n^2 I)^{-1} \mathbf{y} \tag{8.6}$$

$$\mathrm{Var}\left[\hat{f}(\mathbf{x}^*)\right] = k(\mathbf{x}^*, \mathbf{x}^*) - \mathbf{k}_*^T (K + \sigma_n^2 I)^{-1} \mathbf{k}_* \tag{8.7}$$

mit $\mathbf{k}_* = [k(\mathbf{x}^*, \mathbf{x}_1), \ldots, k(\mathbf{x}^*, \mathbf{x}_N)]^T$.

Der Erwartungswert (Gl. 8.6) ist die Punktvorhersage; die Varianz (Gl. 8.7) quantifiziert die Vorhersageunsicherheit. Datendichte Bereiche ⇒ geringe Varianz; datenleere Bereiche ⇒ große Varianz.

8.5 Hyperparameter-Optimierung via Marginal Likelihood

Kernelparameter (ℓ, σ_f^2, σ_n^2) werden durch Maximierung der *Log-Marginal-Likelihood* geschätzt [**rasmussen2006**]:

$$\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) = -\frac{1}{2}\mathbf{y}^T K_y^{-1} \mathbf{y} - \frac{1}{2}\log|K_y| - \frac{N}{2}\log(2\pi) \tag{8.8}$$

mit $K_y = K + \sigma_n^2 I$. Das LMNTool führt `n_restarts_optimizer` Neustarts der L-BFGS-B-Optimierung durch.

8.6 Antwortfläche

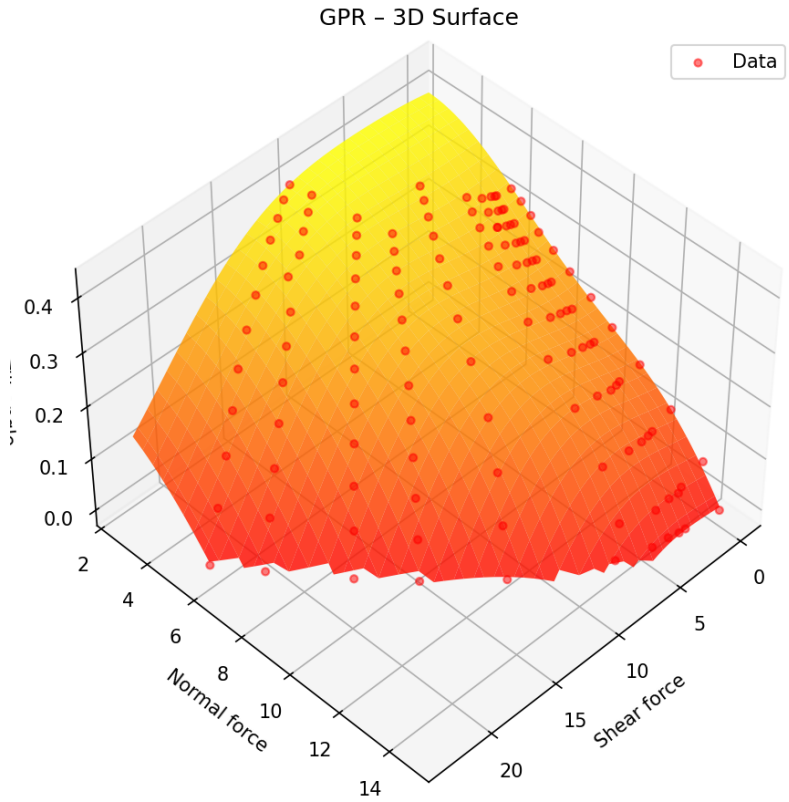


Abbildung 8.1: Modellierte Antwortfläche (GPR). Gaussian Process Regression liefert eine sehr glatte Approximation, die eng den Trainingsdaten folgt. In datenleeren Randbereichen zieht das Modell in Richtung der Priorfunktion.

8.7 Hyperparameter von GaussianProcessRegressor

Tabelle 8.1: Wichtige Hyperparameter der GPR im LMNTool

Parameter	**Typ**	**Beschreibung**
`length_scale` (ℓ)	float	Längenskalierungsparameter des Kernels.
`noise_level` (σ_n^2)	float	Rauschterm im Kernel (Messrauschen).
`n_restarts_optimizer`	int	Neustarts der Marginal-Likelihood-Optimierung.
`max_fit_samples`	int	Max. Datenpunkte für Training (Skalierungsbegrenzung).

8.8 Stärken und Schwächen von GPR

Tabelle 8.2: Stärken und Schwächen der GPR

Stärken	**Schwächen**
Fundierte probabilistische Basis.	Kubische Rechenkosten $\mathcal{O}(N^3)$.
Vorhersageunsicherheit direkt verfügbar.	Stark kernelabhängig.
Gute Güte bei kleinen Datensätzen.	Nicht skalierbar für sehr große N.
Kernelparameter automatisch aus Daten geschätzt.	Numerische Instabilität möglich.

Kapitel 9

LMNTool Multi GUI

9.1 Überblick und Architektur

LMNTool Multi GUI ist ein Python-basiertes Vergleichswerkzeug für fünf Regressionsverfahren auf einem einheitlichen Datensatz. Das Tool ist eine Portierung und Erweiterung der MATLAB LMNTool-Toolbox [**matlab_lmnhelp**] und verwendet für HILOMOT/LOLIMOT eine Eigenimplementierung sowie für MLP, SVR und GPR scikit-learn [**pedregosa2011**].

9.2 Datenimport und Vorverarbeitung

CSV-Import mit Spaltenreihenfolge y, x_1, x_2. Nach dem Import:

- Prüfung auf numerische Typen,
- optionale Normierung auf $[0, 1]$ oder $\mathcal{N}(0, 1)$,
- 3D-Datenvorschau zur Plausibilitätsprüfung.

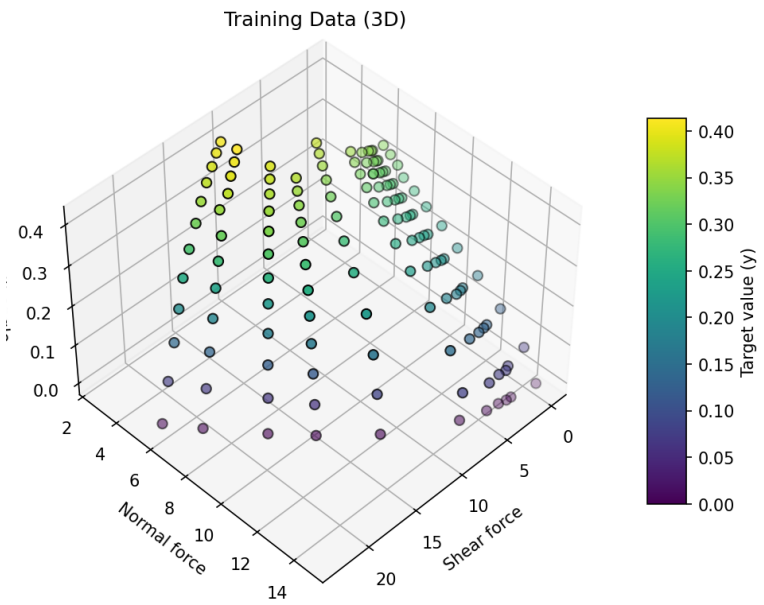


Abbildung 9.1: 3D-Datenvorschau im LMNTool nach dem CSV-Import. Die Visualisierung ermöglicht eine erste visuelle Plausibilitätsprüfung – z. B. auf Ausreißer, Lücken oder unerwartete Strukturen.

9.3 Methoden-Tabs und Parametrierung

Jeder Methoden-Tab enthält Eingabefelder für alle methodenspezifischen Hyperparameter (vgl. Tab. 9.1), einen `Train`-Button und eine Statusanzeige mit MSE, MAE, R^2 nach dem Training.

Tabelle 9.1: Zusammenfassung der Kernparameter aller Methoden im LMNTool

Methode	**Typische Kernparameter**
HILOMOT	`max_leafs`, `min_samples_per_leaf`, `smoothness`, `demanded_membership_value`, `oblique`
LOLIMOT	Analoge Baumparameter ohne oblique
MLP	`hidden_layer_sizes`, `alpha`, `learning_rate_init`, `max_iter`
SVR	`C`, `gamma`, `epsilon`
GPR	`length_scale`, `noise_level`, `n_restarts_optimizer`, `max_fit_samples`

9.4 Training und Bewertungslogik

Trainingsmodi:

- **Einzeltraining:** Nur die aktive Methode wird trainiert.
- **Train-All:** Alle fünf Methoden nacheinander mit aktuellen Parametern.

Bewertungslogik (vier normierte Kriterien):

$$\text{Score}_{\text{gesamt}} = w_1 S_{\text{Acc}} + w_2 S_{\text{Ext}} + w_3 S_{\text{Eff}} + w_4 S_{\text{Int}}, \quad \sum w_i = 1 \tag{9.1}$$

Zusätzlich werden berechnet: Extrapolation Linearity, Numerical Efficiency, Interpretability Score.

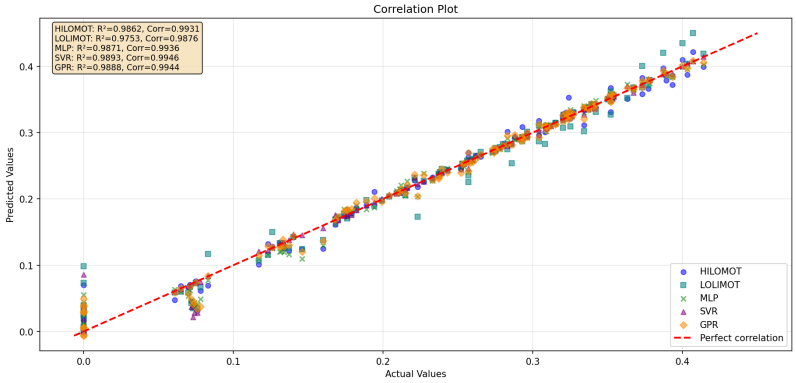


Abbildung 9.2: Korrelationsplot im LMNTool: gemessene Zielwerte y vs. Modellvorhersage $\hat{y}$. Gute Regression: Punkte nahe der Winkelhalbierenden. Systematische Abweichungen zeigen Modellbias.

Hinweis

Die normalisierte Vergleichsansicht skaliert jedes Kriterium auf $[0, 1]$, wobei 1 jeweils dem besten Methodenwert entspricht.

9.5 Visualisierungen

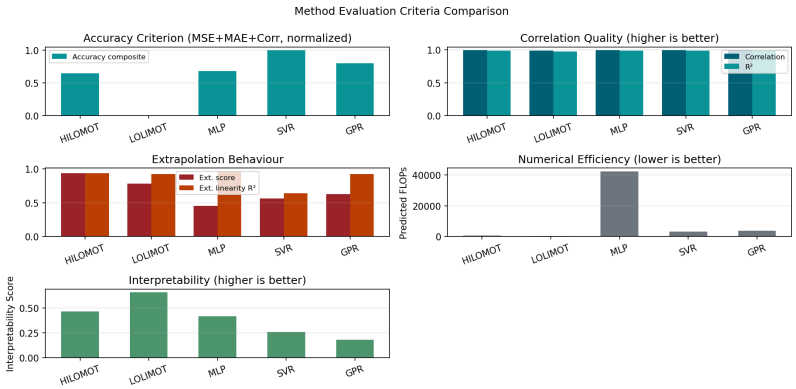


Abbildung 9.3: Evaluation Bars: absoluter Vergleich aller fünf Methoden über die Kriterien Accuracy, Extrapolation Linearity, Numerical Efficiency und Interpretability.

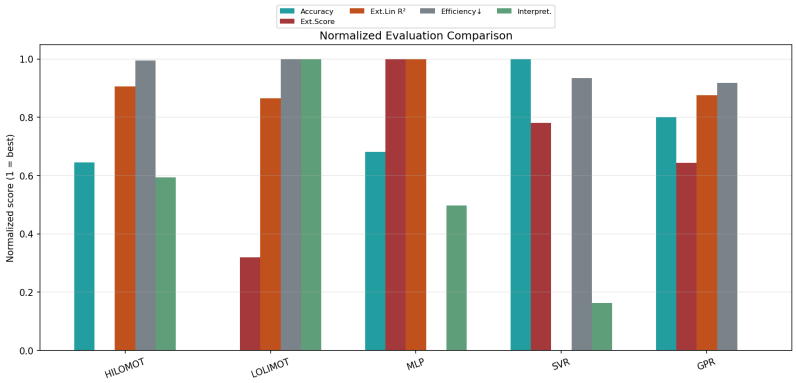
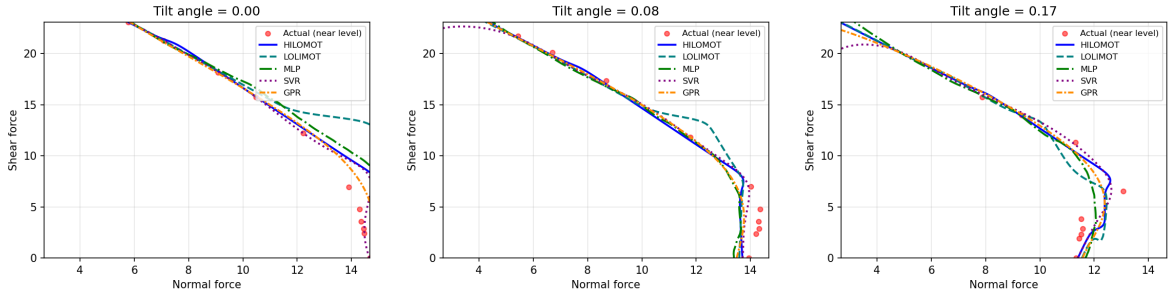
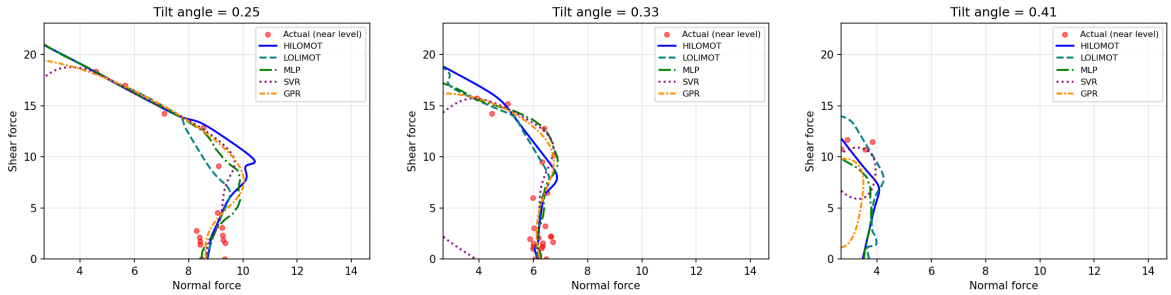


Abbildung 9.4: Normalisierte Evaluation Bars: Jedes Kriterium ist auf $[0, 1]$ skaliert, wobei 1 der jeweils besten Methode entspricht. Diese Darstellung macht relative Stärken/Schwächen auf einen Blick erkennbar.



(a) Schnitt 1 (b) Schnitt 2 (c) Schnitt 3



(d) Schnitt 4 (e) Schnitt 5 (f) Schnitt 6

Abbildung 9.5: Tilt Angle Cuts: Schnitte durch die modellierten Antwortflächen bei sechs verschiedenen Normalkraft-Niveaus ($x_2 = \text{const}$). Alle fünf Methoden werden je Schnitt überlagert dargestellt. Abweichungen zwischen den Kurven zeigen methodenspezifische Unterschiede in Interpolation und Extrapolationsverhalten.

9.6 Report- und Datenexport

Exportformate:

- **JSON:** Vollständige Modellparameter aller trainierten Methoden.
- **CSV:** Vorhersagen $\hat{y}$ aller Methoden.
- **TXT:** Lesbare Zusammenfassung der Gütekriterien.

- **LaTeX/PDF:** Automatisch generierter Bericht mit Tabellen und Abbildungen.

Für HILOMOT/LOLIMOT: Dedizierter Parameterexport mit allen Baumknoten für direkte Einbettung in Echtzeitsysteme (z. B. MATLAB/Simulink).

Kapitel 10

MeinWindowLMNTool

10.1 Überblick und Einordnung

MeinWindowLMNTool ist eine interaktive, MATLAB-basierte Benutzeroberfläche für die Identifikation nichtlinearer Systeme mittels lokaler Modellnetze. Sie baut direkt auf der MATLAB LMNTool-Toolbox (Version 1.5.2.1) der Universität Siegen auf [**matlab_lmnhelp**, **matlab_lmn**] und kapselt deren Kernfunktionalität – HILOMOT- und LOLIMOT-Training, Visualisierung sowie Parameterexport – in einer *App-Designer*-Anwendung mit einer einzigen, übersichtlichen Fensteroberfläche.

Merksatz

Einordnung: Während das *LMNTool Multi GUI* (Kapitel 9) ein Python-basiertes Vergleichswerkzeug für fünf Regressionsverfahren ist, fokussiert *MeinWindowLMNTool* vollständig auf HILOMOT und LOLIMOT im MATLAB-Ökosystem. Es richtet sich primär an Anwender, die bereits mit MATLAB vertraut sind oder eine enge Integration mit Simulink/Control-Toolboxen benötigen.

Tabelle 10.1: Gegenüberstellung: MeinWindowLMNTool vs. LMNTool Multi GUI

Merkmal	**MeinWindowLMNTool**	**LMNTool Multi GUI**
Sprache / Plattform	MATLAB (App Designer)	Python (tkinter/matplotlib)
Unterstützte Methoden	HILOMOT, LOLIMOT	HILOMOT, LOLIMOT, MLP, SVR, GPR
Zielgruppe	MATLAB-Nutzer, Simulink-Integration	Plattformübergreifend, Open-Source
Lizenz	MATLAB-Lizenz erforderlich	Python, kostenlos
Export	`.mat`, JSON, Text	JSON, CSV, TXT, LaTeX/PDF
Unsicherheitsangabe	Nein	Nur über GPR

10.2 Programmaufbau und Startvoraussetzungen

Voraussetzungen:

- MATLAB R2020b oder neuer (App Designer erst ab R2016a verfügbar).
- LMNTool-Toolbox v1.5.2.1 [**matlab_lmn**] im MATLAB-Suchpfad (`addpath`).
- Statistics and Machine Learning Toolbox (für Basisfunktionen).

Starten der Anwendung:

```
% Option 1: Direkt aus dem MATLAB Command Window
run('MeinWindowLMNTool.mlapp')

% Option 2: Doppelklick auf MeinWindowLMNTool.mlapp im
%            MATLAB Current Folder Browser
```

Listing 10.1: MeinWindowLMNTool starten

Nach dem Start öffnet sich das Hauptfenster mit drei Hauptbereichen:

1. **Steuerbereich (links):** Dateiladen, Methoden- und Parameterwahl, Trainingsstart.
2. **Plotbereich (Mitte/rechts):** Visualisierungen (3D-Fläche, Partitionierungskarte, Korrelationsplot, Schnittdarstellungen).
3. **Ergebnisbereich (unten):** Gütekennzahlen, Modellzusammenfassung und Exportbuttons.

10.3 Datenimport

MeinWindowLMNTool akzeptiert Datensätze als CSV-Datei im selben 3-spaltigen Format wie das Python-Tool:

- Spalte 1: Zielgröße y (z. B. Tilt Angle),
- Spalte 2: Eingangsgröße x_1 (z. B. Shear Force),
- Spalte 3: Eingangsgröße x_2 (z. B. Normal Force).

Trennzeichen werden automatisch erkannt (`;`, `,`, Tab). Zusätzlich können Daten direkt als MATLAB-Variable (`.mat`-Datei) übergeben werden.

```
% Trainingsdaten laden
data = readmatrix('tiltangle_data.csv');
y  = data(:,1);   % Tilt Angle
x1 = data(:,2);   % Shear Force
```

```
x2 = data(:,3);    % Normal Force

% Datenmenge X fuer LMNTool
X_train = [x1, x2];
```

Listing 10.2: Daten programmatisch laden (Skript-Modus)

Nach dem Import zeigt das Tool eine 3D-Vorschau der Rohdaten analog zu Abbildung 1.1 (Kapitel 1), um Ausreißer und Lücken visuell zu prüfen, bevor das Training gestartet wird.

10.4 Methoden- und Parameterwahl

Im Steuerbereich wählt der Anwender über ein `uiswitch`-Element zwischen **LOLIMOT** (achsenparallel) und **HILOMOT** (oblique Splits). Alle relevanten Hyperparameter können über Spinner-Felder (`uispinner`) und Checkboxen (`uicheckbox`) eingestellt werden.

Tabelle 10.2: Trainingsparameter in MeinWindowLMNTool

Parameter	**Typ**	**Standard**	**Beschreibung**
Max Leaves	int	15	Maximale Blätteranzahl (Modellkomplexität).
Min Samples per Leaf	int	6	Mindestanzahl effektiver Trainingspunkte je Blatt.
Smoothness (ρ)	float	0,8	Überlappung der Sigmoid-Gültigkeitsregionen.
Membership Value (μ^*)	float	0,4	Mindest-Gültigkeitswert für WLS-Einbeziehung.
Oblique Splits	bool	an	`an` = HILOMOT; `aus` = LOLIMOT.
Kreuzvalidierung	bool	aus	Aktiviert k-fache CV zur Strukturwahl.
k (CV-Folds)	int	5	Anzahl der Folds bei Kreuzvalidierung.

Hinweis

Empfehlung: Für komplexe, diagonale Datenstrukturen (wie den Tilt-Angle-Datensatz) empfiehlt sich `Oblique Splits = an` (HILOMOT) mit `Max Leaves = 10-15` und `Smoothness = 0,4-0,8`. Für achsenparallele, weniger gekoppelte Daten reicht LOLIMOT oft schon mit `Max Leaves = 8-12`.

10.5 Training

Das Training wird durch Klick auf den **Train**-Button ausgelöst. Der Trainingsfortschritt wird in einer Fortschrittsanzeige (`uiprogressdlg`) angezeigt, sodass die Oberfläche während des Baumwachstums reaktionsfähig bleibt.

Intern ruft die App die LMNTool-Kernfunktion auf:

```
% HILOMOT-Training (intern in der App-Callback-Funktion)
opt = lmntool_options();
opt.maxLeafs       = app.MaxLeavesSpinner.Value;
opt.smoothness     = app.SmoothnessSpinner.Value;
opt.oblique        = app.ObliqueSwitchValue;
opt.minSamplesLeaf = app.MinSamplesSpinner.Value;
opt.membershipVal  = app.MembershipSpinner.Value;

model = hilomot(X_train, y, opt);   % LMNTool-Funktion
```

Listing 10.3: Interner Aufruf der LMNTool-Trainingsfunktion

Nach dem Training werden direkt im Ergebnisbereich ausgegeben:

Metrik	**Formel**	**Beschreibung**
MSE	$\frac{1}{N}\sum(y_i - \hat{y}_i)^2$	Mittlerer quadratischer Fehler
MAE	$\frac{1}{N}\sum\lvert y_i - \hat{y}_i\rvert$	Mittlerer absoluter Fehler
RMSE	$\sqrt{\text{MSE}}$	Wurzel des MSE (in Einheiten von y)
R^2	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Bestimmtheitsmaß
Lokale Modelle	M	Anzahl Blätter im Baum
Trainingszeit	t [s]	Rechenaufwand des Baumwachstums

10.6 Visualisierungen

MeinWindowLMNTool stellt dieselben zentralen Visualisierungen bereit, die auch aus dem Python-Tool bekannt sind (Kapitel 5.1, 9.2):

Tabelle 10.3: Visualisierungstypen in MeinWindowLMNTool

Plot-Typ	**Beschreibung**
3D Surface	Vorhersagefläche $\hat{y}(x_1, x_2)$ mit eingebetteten Trainingspunkten (MATLAB `surf` + `scatter3`).
Partition Map	2D-Partitionierungsplot mit eingezeichneten Trennhyperebenen und farbcodierten Gültigkeitsbereichen (`pcolor`).
Correlation Plot	Streudiagramm y vs. $\hat{y}$ mit Winkelhalbierender zur Diagnose von Bias.
Tilt Cuts	Schnitte $\hat{y}(x_1)$ bei verschiedenen Niveaus $x_2 = \text{const.}$ Direkt vergleichbar mit Abb. 9.5.
Validity Functions	2D-Darstellung der einzelnen Gültigkeitsfunktionen $\Phi_i(x_1, x_2)$ für jedes Blatt. Einzigartig unter den beiden Tools.

Ein besonderes Alleinstellungsmerkmal von *MeinWindowLMNTool* gegenüber dem Python-

Pendant ist die **Validity-Function-Visualisierung**: Für jedes Blatt i wird $\Phi_i(x_1, x_2)$ als Oberfläche geplottet (Gleichung 3.4). Diese Darstellung macht transparent, welche Datenpunkte durch welches lokale Modell beschrieben werden – ein wesentlicher Beitrag zur Interpretierbarkeit.

```
% Eingangsraster
[x1g, x2g] = meshgrid(linspace(min(x1),max(x1),60), ...
                      linspace(min(x2),max(x2),60));
Xgrid = [x1g(:), x2g(:)];

% Gueltigkeitsgewichte aus dem Modell abrufen
Phi = lmntool_validity(model, Xgrid);   % N_grid x M Matrix

% Blatt i plotten
figure;
surf(x1g, x2g, reshape(Phi(:,i), size(x1g)));
xlabel('Shear Force'); ylabel('Normal Force');
zlabel(['\Phi_' num2str(i)]);
title(['Gueltigkeitsfunktion -- Blatt ' num2str(i)]);
```

Listing 10.4: Validity-Function-Plot für Blatt i (Skript-Modus)

10.7 Export und Integration

MeinWindowLMNTool bietet folgende Exportoptionen:

Tabelle 10.4: Exportformate in MeinWindowLMNTool

Format	Inhalt
`.mat`	Vollständige MATLAB-Modellstruktur: Baum, Knotenparameter, Kappa-Werte, Gewichte. Direkt ladbar mit `load('model.mat')`.
JSON	Maschinenlesbare Beschreibung aller Baumknoten (kompatibel mit Python-Tool).
Text	Lesbare Zusammenfassung: Metriken, Baumstruktur, lokale Modellkoeffizienten.
Simulink-Block	Erzeugung eines parametrisierten `Interpreted MATLAB Function`-Blocks für direkte Einbettung in Simulink-Modelle.

Simulink-Integration: Der automatisch erzeugte Simulink-Block übernimmt die gelernten Modellparameter (Gewichte, Bias, Kappa) und bildet die LMN-Vorhersagefunktion (Gleichung 3.1) direkt in Echtzeit ab. Dies ermöglicht die direkte Nutzung des trainierten Modells in Hardware-in-the-Loop (HiL)-Umgebungen oder Steuergeräte-Prototypen.

```
% Gespeichertes Modell laden
load('hilomot_tiltangle_model.mat', 'model');

```

```
% Vorhersage fuer neuen Datenpunkt
x_new = [0.35, 1.2];   % [Shear Force, Normal Force]
y_hat = lmntool_predict(model, x_new);

fprintf('Vorhergesagter Tilt Angle: %.4f\n', y_hat);
```

Listing 10.5: Modell nach Training laden und auswerten

10.8 Vergleich mit dem Python LMNTool Multi GUI

Beide Werkzeuge ergänzen sich sinnvoll und richten sich an unterschiedliche Anwendergruppen:

MeinWindowLMNTool (MATLAB): Ideal für MATLAB-Nutzer in Industrie und Hochschule, besonders wenn eine direkte Simulink-Integration oder Anbindung an MATLAB-Workflows gewünscht ist. Stärken: Validity-Function-Visualisierung, Simulink-Export, native MATLAB-Datenstrukturen.

LMNTool Multi GUI (Python): Ideal für plattformübergreifenden Einsatz und wenn auch MLP, SVR und GPR verglichen werden sollen. Stärken: Mehr Methoden, normierter Mehrkriterienscore, kostenlos nutzbar.

Tabelle 10.5: Funktionsumfang im Direktvergleich

Funktion	**MeinWindowLMNTool**	**LMNTool Multi GUI**
HILOMOT-Training	✓	✓
LOLIMOT-Training	✓	✓
MLP / SVR / GPR	–	✓
3D Antwortfläche	✓	✓
Partitionierungskarte	✓	✓
Korrelationsplot	✓	✓
Tilt Angle Cuts	✓	✓
Validity-Functions-Plot	✓	–
Kreuzvalidierung (GUI)	✓	–
Mehrkriterienscore	–	✓
Simulink-Export	✓	–
JSON-Export	✓	✓
`.mat`-Export	✓	–
LaTeX/PDF-Report	–	✓
MATLAB-Lizenz erforderlich	Ja	Nein

Merksatz

Für reine HILOMOT/LOLIMOT-Anwendungen in MATLAB-Umgebungen ist *MeinWindowLMNTool* die direktere Wahl. Für einen übergreifenden Methodenvergleich (inkl. MLP, SVR, GPR) bietet das *LMNTool Multi GUI* den vollständigeren Rahmen.

Kapitel 11

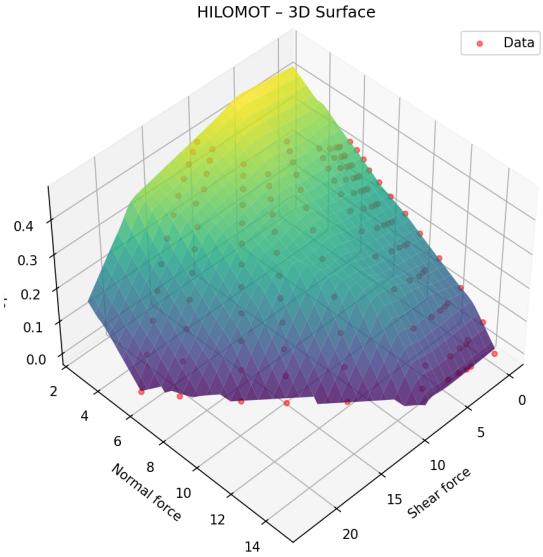
Vergleich der Methoden

11.1 Methodenüberblick

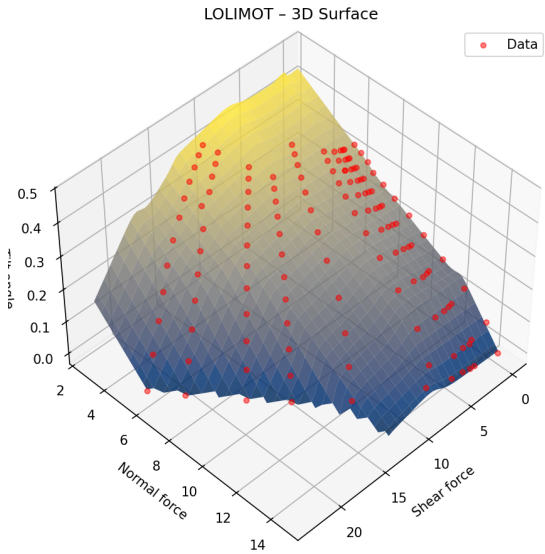
Tabelle 11.1: Qualitative Einordnung der fünf Methoden nach Hauptcharakteristika

Methode	**Modelltyp**	**Kernidee**	**Interpret.**	**Unsich.**
HILOMOT	Lok. Modellnetz	Oblique Splits	Hoch	Nein
LOLIMOT	Lok. Modellnetz	Achsenparallele Splits	Sehr hoch	Nein
MLP	Neuronales Netz	Globale Approx.	Gering	Nein
SVR	Kernel-Methode	ε-Margin	Mittel	Nein
GPR	Probabilistisch	Bayes'sche Inferenz	Mittel	Ja

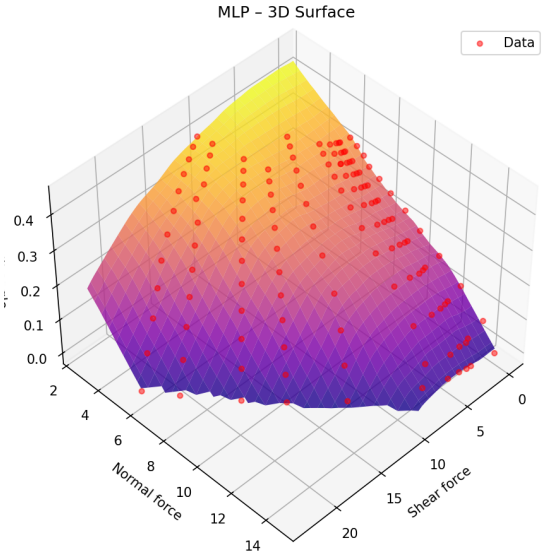
11.2 Vergleich der Antwortflächen



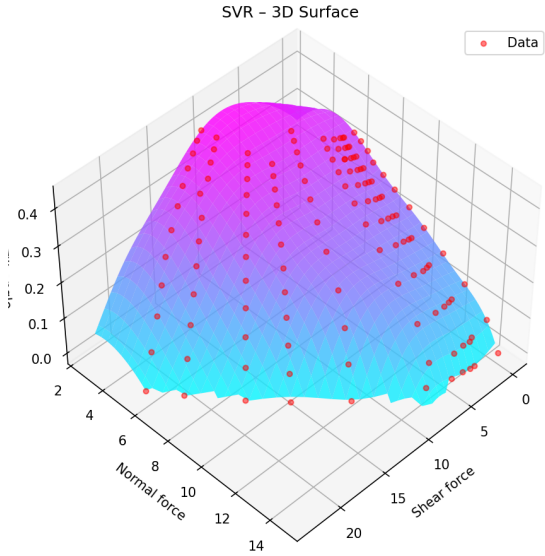
(a) HILOMOT



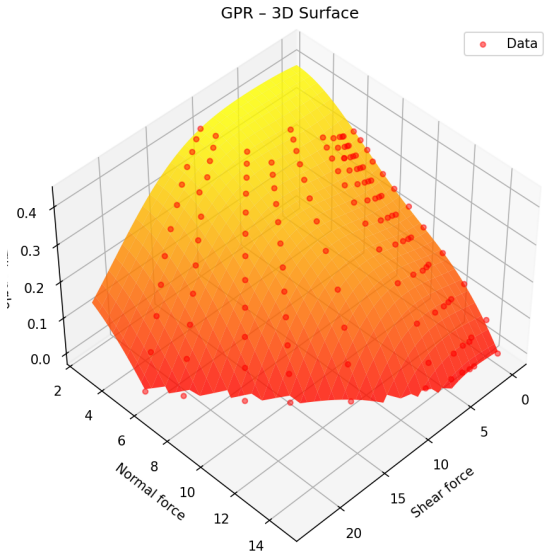
(b) LOLIMOT



(c) MLP



(d) SVR



(e) GPR

Abbildung 11.1: Vergleich der modellierten Antwortflächen aller fünf Methoden auf demselben Datensatz. Qualitative Unterschiede in Glätte, Partitionierungsstruktur und globalem

11.3 Tabellarischer Vergleich

Tabelle 11.2: Ausführlicher Vergleich nach Stärken und Schwächen

Methode	**Stärken**	**Schwächen**
HILOMOT	Gute Balance Genauigkeit/Interpretierbarkeit; oblique Splits effizient; physikalisch plausible lokale Modelle; Parameterexport.	Mehr Parameter als globale Modelle; greedy; höherer Tuningaufwand bei Rauschen.
LOLIMOT	Sehr hohe Transparenz; robuste Modellbildung; einfach kommunizierbar; gering Rechenaufwand.	Treppenstufen bei diagonalen Grenzen; mehr Blätter bei gekoppelten Strukturen.
MLP	Hohe Flexibilität; starke Approximation; gute Fit-Werte bei genügend Daten.	Black Box; sensitiv auf Hyperparameter; Overfitting-Risiko.
SVR	Gute Generalisierung; robustes Lernprinzip; RBF-Kernel effektiv.	Parameter schwer einzustellen; hohe Laufzeiten bei großem N; keine Unsicherheit.
GPR	Probabilistische Basis; Unsicherheitsquantifizierung; gut bei kleinen Datensätzen.	$\mathcal{O}(N^3)$ Kosten; kernelabhängig; nicht skalierbar.

11.4 Rechenkomplexität

Tabelle 11.3: Ungefähre Rechenkomplexität der fünf Methoden

Methode	**Training**	**Vorhersage**
HILOMOT/LOLIMOT	$\mathcal{O}(M \cdot n \cdot N)$	$\mathcal{O}(M)$
MLP	$\mathcal{O}(\text{Iter} \cdot N \cdot P)$	$\mathcal{O}(P)$
SVR	$\mathcal{O}(N^2)$–$\mathcal{O}(N^3)$	$\mathcal{O}(\lvert\text{SV}\rvert)$
GPR	$\mathcal{O}(N^3)$	$\mathcal{O}(N^2)$

M: Blätter, n: Dim., N: Datenpunkte, P: Netzparameter, SV: Support Vectors

11.5 Anwendungsempfehlungen

1. **Physikalische Interpretierbarkeit:** HILOMOT oder LOLIMOT.
2. **Diagonale/schräge Trennstrukturen:** HILOMOT mit oblique Splits.
3. **Achsenparallele Strukturen:** LOLIMOT.

4. **Reine Vorhersagegüte, ausreichende Daten:** MLP oder SVR.
5. **Kleiner Datensatz, Unsicherheit gewünscht:** GPR.
6. **Echtzeiteinsatz:** HILOMOT/LOLIMOT (expliziter Parameterexport).
7. **Robuste Entscheidung:** Mehrkriteriensicht und Kreuzvalidierung nutzen.

Merksatz

Es gibt keine universell beste Methode. Die optimale Wahl hängt von Datencharakteristik, Erklärbarkeitsanforderung, Rechenbudget, Datenmenge und Extrapolationsanforderung ab. Der normalisierte Balkenvergleich (Abb. 9.4) zeigt diese Abwägungen direkt.

Kapitel 12

Diskussion

12.1 Interpretation der Ergebnisse

Am verwendeten Datensatz (Tilt Angle über Shear Force und Normal Force) zeigen alle fünf Methoden grundstätzlich eine gute Approximationsleistung, was auf eine glatte, gut lernbare Datenstruktur hindeutet. Dennoch lassen sich methodenspezifische Unterschiede beobachten:

HILOMOT und LOLIMOT bilden die Struktur durch explizite Partitionierung ab. Die Partitionierungskarte (Abb. 5.1b) zeigt klar, welche Teilmodelle dominieren. Bei HILOMOT sind schräge Trennlinien erkennbar, die auf nicht-achsenparallele Datenstruktur hinweisen. LOLIMOT benötigt für ähnliche Genauigkeit typischerweise mehr Blätter.

MLP liefert eine glatte, global definierte Antwortfläche (Abb. 6.1) ohne erkennbare Partitionierungsstruktur. Das Modell ist leistungsfähig, bietet aber keinen Einblick in die Datenstruktur.

SVR produziert eine glatte Oberfläche (Abb. 7.1) mit Robustheit gegenüber Ausreißern durch den ε-insensitiven Verlust.

GPR zeigt die glatteste Approximation (Abb. 8.1) und folgt den Daten eng in dicht besetzten Regionen. Die Vorhersageunsicherheit ist ein wichtiger Vorteil bei sicherheitsrelevanten Anwendungen.

Die Tilt-Angle-Schnittdarstellungen (Abb. 9.5) zeigen, dass alle Methoden im Trainingsbereich ähnliche Verläufe haben. Unterschiede treten primär in Randbereichen und bei der Extrapolation auf.

12.2 Kritische Betrachtung der Hyperparameterwahl

Alle Methoden weisen eine erhebliche Empfindlichkeit gegenüber der Hyperparameterwahl auf:

- HILOMOT/LOLIMOT: `max_leafs` und `smoothness` entscheiden über den Bias-Varianz-Kompromiss. Zu viele Blätter führen zu Overfitting.
- MLP: Netzgröße, Lernrate und α sind entscheidend; Grid-Search aufwändig.
- SVR: C, γ und ε interagieren stark; MATLAB empfiehlt Kreuzvalidierung [**matlab_svr**].
- GPR: Kernelparameter werden automatisch geschätzt, Anzahl Neustarts beeinflusst die Qualität.

12.3 Generalisierungsfähigkeit und Extrapolation

Das Verhalten außerhalb des Trainingsbereichs (Extrapolation):

HILOMOT/LOLIMOT: Extrapolation über lokale lineare Modelle in Randbereichen – oft linear und physikalisch interpretierbar.

MLP: Unvorhersehbare Extrapolation; Netze können außerhalb stark oszillieren.

SVR: RBF-Kernel läuft außerhalb gegen null – unerwartete Vorhersagen möglich.

GPR: Zieht gegen die Priorfunktion; Unsicherheit wächst – transparentes "Nicht-Wissen".

12.4 Grenzen des Vergleichs

- Vergleich basiert auf einem Datensatz – andere Datensätze können zu anderen Rangfolgen führen.
- Hyperparameter wurden nicht systematisch per Grid-Search optimiert.
- Tiefere neuronale Netze (CNN, Transformer) wurden nicht berücksichtigt.
- Interpretierbarkeit und Extrapolation sind qualitativ eingestuft.

Kapitel 13

Zusammenfassung

13.1 Wichtigste Erkenntnisse

Diese Dokumentation hat fünf Regressionsverfahren von den mathematischen Grundlagen bis zur Implementierung im LMNTool Multi GUI ausführlich beschrieben. Die zentralen Erkenntnisse:

1. **Lokale Modellnetze (HILOMOT, LOLIMOT)** bieten eine einzigartige Kombination aus nichtlinearer Approximationsleistung und physikalischer Interpretierbarkeit.
2. **HILOMOT** erreicht bei gekoppelten Daten durch oblique Splits oft bessere Ergebnisse mit weniger Blättern als LOLIMOT.
3. **MLP** ist leistungsfähig bei ausreichenden Daten, bietet aber wenig Einblick in die Datenstruktur.
4. **SVR** ist robust und gut generalisierend für kleine bis mittlere Datensätze.
5. **GPR** liefert als einzige Methode Vorhersageunsicherheiten – ein wesentlicher Vorteil für sicherheitskritische Anwendungen. Die kubische Skalierung limitiert den Einsatz bei großen Datensätzen.
6. **Das LMNTool Multi GUI** ermöglicht einen direkten, reproduzierbaren Vergleich aller fünf Methoden mit einheitlichen Metriken, Multi-Kriterien-Bewertung und interaktiven Visualisierungen.

13.2 Praktische Handlungsempfehlungen

Kleiner Datensatz ($N < 500$), Unsicherheit wichtig: GPR.

Kleiner bis mittlerer Datensatz, keine Interpretierbarkeit nötig: SVR.

Mittlerer bis großer Datensatz, Black Box akzeptiert: MLP.

Interpretierbarkeit wichtig, achsenparallele Strukturen: LOLIMOT.

Interpretierbarkeit wichtig, gekoppelte Eingänge: HILOMOT.

Echtzeiteinsatz in Steuergerät: HILOMOT oder LOLIMOT (Parameterexport).

13.3 Ausblick

Zukünftige Erweiterungen könnten umfassen:

- Unterstützung höherer Eingangsdimension ($n > 2$).
- Integration tieferer neuronaler Netze als sechste Methode.
- Approximate GPR (Sparse GP) für große Datensätze.
- Automatische Hyperparameter-Optimierung (AutoML, Bayesian Optimization).
- Bootstrap-Konfidenzintervalle für Methoden ohne probabilistischen Rahmen.